



WORK in TECH



GUIA PRÁTICO · APOSTILA DA WEB

Dicionário dos Web Fundamentos

HTML · **CSS** · **JavaScript** — do esqueleto ao cérebro da página

HTML · ESQUELETO + CSS · PELE E MÚSCULOS + JS · CÉREBRO

Uma página web é como um corpo humano

Todo site que você visita — de uma rede social a uma loja online — é construído com **três linguagens** trabalhando juntas. Cada uma tem um papel bem definido, e entender essa divisão é o primeiro passo para programar para a web.

HTML

O esqueleto

Dá a **estrutura**: diz o que é título, texto, imagem, botão...

CSS

Pele e músculos

Dá a **aparência**: cores, fontes, tamanhos, posições, animações.

JAVASCRIPT

O cérebro

Dá o **comportamento**: pensa, decide e reage às ações.

Sumário

Capítulo 1 — HTML: o esqueleto da página 3

- Estrutura básica de um documento 4
- Semântica: o layout com significado 5
- Tags HTML: o ranking completo (113 tags) 6
- Formulários em ação + tipos de input 11
- Atributos HTML: o ranking completo (84 atributos) 12

Capítulo 2 — CSS: a pele e os músculos 16

- Como aplicar o CSS e como a cascata funciona 17
- Box Model, unidades de medida e especificidade 18
- Seletores CSS: do mais usado ao mais avançado 19
- Propriedades CSS em 8 níveis de dificuldade 20
- Cores em hexadecimal (página especial) 25

Capítulo 3 — JavaScript: o cérebro da página 26

- O DOM, como incluir o JS e o Console 27
- Declaração de variáveis: var, let e const 28
- Tipos de dados, typeof e conversões 29
- Operadores: aritméticos, comparação e lógicos 30
- Códigos JS essenciais (do mais importante ao mais avançado) 31
- Lista de eventos mais usados 38
- Projeto prático: lista de tarefas com HTML + CSS + JS 39

Como ler os cards

#01 `<div>` ESTRUTURA

Caixa genérica sem significado próprio. É o "bloco de montar" mais usado.

```
<div class="card">...</div>
```

- **#01** — posição no ranking: dos mais usados / básicos para os menos usados / complexos.
- **Nome** — a tag, atributo, propriedade ou código.
- **Etiqueta** — a categoria (texto, formulário, layout...).
- **Descrição** — o que faz, em poucas palavras.
- **Caixinha cinza** — um exemplo real de uso, colorido como no editor.

Cores de cada capítulo

- **Laranja** — páginas de HTML
- **Azul-escuro** — páginas de CSS
- **Amarelo** — páginas de JavaScript

Dica de estudo: leia um card, digite o exemplo no VS Code, abra no navegador e mude alguma coisa. Errar faz parte — é assim que o cérebro aprende!

Dica: Esqueleto (HTML) + Músculos e pele (CSS) + Cérebro (JS) = uma página viva!

CAPÍTULO 1

HTML

O esqueleto da página

HTML (*HyperText Markup Language* — Linguagem de Marcação de Hipertexto) é quem dá **estrutura** a qualquer página da web. Assim como o esqueleto sustenta o corpo, o HTML diz **o que é cada coisa**: isto é um título, isto é um parágrafo, aqui vai uma imagem, ali um botão.

Ele funciona com **tags**: etiquetas escritas entre os sinais `< >` que "embrulham" o conteúdo.

```
<p  class="destaque">Olá, mundo! </p>
```

nome da tag | atributo = "valor" | conteúdo | fechamento

Tag de abertura + conteúdo + tag de fechamento = um **ELEMENTO**. A barra **/** indica que a tag está fechando.

Tags vazias

Algumas tags não têm conteúdo, então não fecham: `<img>`, `<br>`, `<input>`, `<meta>`.

Aninhamento

Tags vão umas dentro das outras, como ossos encaixados. Feche na ordem inversa: `<p><b>oi</b></p>`

Navegador = leitor

O navegador lê o HTML de cima para baixo e monta a página. Ele **não** mostra as tags, só o resultado.

Neste capítulo

Estrutura básica de um documento 4

Semântica: o layout com significado 5

Tags HTML: o ranking completo (113 tags) 6

Formulários em ação + tipos de input 11

Atributos HTML: o ranking completo (84 atributos) 12

Estrutura básica de um documento HTML

Toda página HTML segue o mesmo "esqueleto" básico. Decore este molde: ele é o ponto de partida de **qualquer** site. Cada número indica uma parte explicada logo abaixo.

INDEX.HTML

```
<!DOCTYPE html> 1
<html lang="pt-BR"> 2
  <head> 3
    <meta charset="UTF-8"> 4
    <meta name="viewport" content="width=device-width, initial-scale=1.0"> 5
    <title>Minha primeira página</title> 6
    <link rel="stylesheet" href="style.css"> 7
  </head>
  <body> 8
    <h1>Olá, mundo!</h1>
    <p>Este é o meu primeiro site.</p>
    <script src="app.js" defer></script> 9
  </body>
</html>
```

1 · `<!DOCTYPE html>`
Avisa ao navegador que é HTML5, a versão moderna. Sempre a primeira linha.

2 · `<html lang="pt-BR">`
A raiz que envolve tudo. O `lang` informa o idioma.

3 · `<head>`
A "cabeça": configurações que não aparecem na tela.

4 · `charset UTF-8`
Faz acentos, ç e emojis aparecerem corretamente.

5 · `viewport`
Faz a página se ajustar à largura do celular.

6 · `<title>`
O texto que aparece na aba do navegador.

7 · `<link>` do CSS
Conecta a pele e os músculos (o arquivo de estilo).

8 · `<body>`
O "corpo": tudo que o usuário vê fica aqui.

9 · `<script defer>`
Conecta o cérebro (o JavaScript). O `defer` faz ele rodar só depois que todo o HTML foi lido.

A árvore da página

As tags formam uma árvore genealógica: **pais**, **filhos** e **irmãos**. Aqui, `head` e `body` são filhos de `html` e irmãos entre si.

```
html
├── head
│   ├── meta
│   ├── title
│   └── link
└── body
    ├── h1
    ├── p
    └── script
```

Atalho de ouro: no VS Code, crie um arquivo `index.html`, digite `!` e aperte `Tab`. O molde inteiro aparece pronto!

Nome dos arquivos: a página inicial de um site quase sempre se chama `index.html`. Use letras minúsculas, sem espaços e sem acentos: `sobre-nos.html` ✓ `Sobre Nós.html` ✗

Dica do Esqueleto: Toda tag que abre precisa fechar: `<p>...</p>`. Exceções (tags vazias): `img`, `br`, `input`, `meta`, `link`, `hr`.

Semântica: dando nome aos ossos

Tags **semânticas** descrevem o **significado** do conteúdo, não só a aparência. Um `<nav>` diz "aqui fica o menu"; um `<div>` não diz nada. Usar semântica melhora o **SEO** (Google entende melhor seu site), a **acessibilidade** (leitores de tela navegam por regiões) e a **organização** do código.

O layout clássico de uma página



O mesmo layout em código

```html
<body>
  <header>
    <h1>Meu Blog</h1>
  </header>
  <nav>
    <a href="/">Início</a>
    <a href="/sobre">Sobre</a>
  </nav>
  <main>
    <section>
      <h2>Últimos posts</h2>
      <article>...</article>
    </section>
  </main>
  <aside>Posts populares</aside>
  <footer>© 2026</footer>
</body>
```

Sopa de div ✗

```html
<div class="topo">
  <div class="menu">...</div>
</div>
<div class="conteudo">...</div>
<div class="rodape">...</div>
```

Semântico ✓

```html
<header>
  <nav>...</nav>
</header>
<main>...</main>
<footer>...</footer>
```

Quem "lê" o seu HTML?

Navegador
Monta a página na tela.

Google
Entende o que é importante e posiciona seu site nas buscas.

Leitor de tela
Lê a página em voz alta para pessoas cegas, pulando por regiões.

Outros devs
Entendem o código só de bater o olho.

Regra prática: antes de usar uma `div`, pergunte-se "existe uma tag que **significa** isso?". Se sim, use-a. A `div` continua ótima para agrupar coisas só por causa do layout (CSS).

Dica do Esqueleto: Indente o código! Cada tag filha um nível para dentro deixa a estrutura visível como um esqueleto.

Tags HTML — da mais usada à menos usada

#01 — ESTRUTURA

Caixa genérica sem significado próprio. É o "bloco de montar" mais usado para agrupar elementos e criar layouts com CSS.

```
<div class="card">...</div>
```

#02 <a> — LINK

Âncora: cria links para outras páginas, arquivos, e-mails ou partes da mesma página, pelo atributo `href`.

```
<a href="contato.html">Fale conosco</a>
```

#03 <p> — TEXTO

Parágrafo de texto. O navegador coloca um espaço automático antes e depois dele.

```
<p>Olá, mundo!</p>
```

#04 — TEXTO

Container de linha (inline) sem significado. Serve para estilizar um pedacinho de texto.

```
<p>Preço: <span class="verde">R$ 10</span></p>
```

#05 <img> — MÍDIA

Insere uma imagem. Não tem fechamento. O `alt` descreve a imagem para leitores de tela e quando ela não carrega.

```
<img src="gato.png" alt="Gato dormindo">
```

#06 <li> — LISTA

Item de lista (list item). Sempre fica dentro de uma `<ul>` ou `<ol>`.

```
<li>Arroz</li>
```

#07 <ul> — LISTA

Lista não ordenada (com bolinhas). Também é a base da maioria dos menus de navegação.

```
<ul><li>Maçã</li><li>Uva</li></ul>
```

#08 <script> — SCRIPT

Insere ou conecta código JavaScript. Com `defer`, o script só roda depois que o HTML carregou.

```
<script src="app.js" defer></script>
```

#09 <link> — METADADOS

Conecta recursos externos, principalmente arquivos CSS e o ícone da aba (favicon). Fica no `<head>`.

```
<link rel="stylesheet" href="style.css">
```

#10 <meta> — METADADOS

Informações sobre a página (charset, descrição, viewport do celular). Não aparece na tela.

```
<meta name="viewport"
  content="width=device-width, initial-scale=1">
```

#11 <html> — ESTRUTURA

Elemento raiz: envolve o documento inteiro. O atributo `lang` informa o idioma.

```
<html lang="pt-BR">...</html>
```

#12 <head> — ESTRUTURA

"Cabeça" invisível do documento: guarda título, metadados, CSS e scripts.

```
<head><title>Minha página</title></head>
```

#13 <body> — ESTRUTURA

Corpo da página: tudo o que o usuário vê na tela mora aqui dentro.

```
<body><h1>Bem-vindo!</h1></body>
```

#14 <title> — METADADOS

Título que aparece na aba do navegador e nos resultados do Google.

```
<title>Loja do Zé</title>
```

#15 <h1> — TEXTO

Título principal. O ideal é ter só um `<h1>` por página (importante para o SEO).

```
<h1>Receitas da Vovó</h1>
```

#16 <h2> — TEXTO

Subtítulo de 2º nível: divide o conteúdo em grandes seções.

```
<h2>Bolos</h2>
```

#17 <h3> — TEXTO

Título de 3º nível: subseções dentro de um `<h2>`.

```
<h3>Bolo de cenoura</h3>
```

#18 <button> — FORMULÁRIO

Botão clicável. Dentro de um `<form>`, por padrão envia os dados (`type="submit"`).

```
<button type="button">Clique aqui</button>
```

#19 <input> — FORMULÁRIO

Campo de entrada de dados. O `type` muda tudo: texto, e-mail, senha, número, data, checkbox...

```
<input type="email" placeholder="Seu e-mail">
```

#20 <form> — FORMULÁRIO

Formulário: agrupa campos e envia os dados para o `action` usando o `method` (GET ou POST).

```
<form action="/cadastro" method="post">
  ...
</form>
```

#21 <label> — FORMULÁRIO

Rótulo de um campo. Ligado pelo `for` ao `id` do input: clicar no texto foca o campo.

```
<label for="nome">Nome</label>
<input id="nome">
```

#22
 — TEXTO

Quebra de linha, sem fechamento. Use dentro de textos (endereços, poemas), não para dar espaço.

```
Rua A, 10<br>São Paulo
```

#23 <style> — METADADOS

Escreve CSS dentro do próprio HTML (estilo interno). Normalmente fica no `<head>`.

```
<style> h1 { color: red; } </style>
```

#24 <nav> — SEMÂNTICA

Bloco de navegação: agrupa os links principais do site (o menu).

```
<nav><a href="/">Início</a></nav>
```

Dica do Esqueleto: Use só um `<h1>` por página e não pule níveis de título (h1 → h2 → h3).

Tags HTML — da mais usada à menos usada

#25 <header> — SEMÂNTICA

Cabeçalho de uma página ou seção: logo, título, menu.

```
<header><h1>Meu Blog</h1></header>
```

#26 <footer> — SEMÂNTICA

Rodapé de uma página ou seção: créditos, contatos, direitos autorais.

```
<footer>© 2026 Work in Tech</footer>
```

#27 <section> — SEMÂNTICA

Seção temática do conteúdo, normalmente com título próprio.

```
<section id="sobre"><h2>Sobre</h2>
</section>
```

#28 <i> — TEXTO

Itálico para termos técnicos ou estrangeiros. Também é muito usada para ícones.

```
<i>Lorem ipsum</i>
```

#29 — TEXTO

Indica importância forte: aparece em negrito e leitores de tela dão ênfase.

```
<strong>Atenção:</strong> não toque!
```

#30 <main> — SEMÂNTICA

Conteúdo principal e único da página. Só deve existir um `<main>` visível.

```
<main>...conteúdo...</main>
```

#31 <article> — SEMÂNTICA

Conteúdo independente, que faz sentido sozinho: post, notícia, card de produto.

```
<article><h2>Notícia</h2><p>...</p>
</article>
```

#32 <ol> — LISTA

Lista ordenada (numerada: 1, 2, 3...). Ideal para passo a passo.

```
<ol><li>Abrir</li><li>Fechar</li></ol>
```

#33 <table> — TABELA

Cria uma tabela de dados, organizada em linhas (`tr`) e células (`td`/`th`).

```
<table><tr><td>A1</td></tr></table>
```

#34 <tr> — TABELA

Linha da tabela (table row). Contém células `td` ou `th`.

```
<tr><td>Ana</td><td>25</td></tr>
```

#35 <td> — TABELA

Célula de dados da tabela (table data).

```
<td>R$ 49,90</td>
```

#36 <th> — TABELA

Célula de cabeçalho da tabela: aparece em negrito e centralizada.

```
<th>Produto</th>
```

#37 <thead> — TABELA

Agrupa a(s) linha(s) de cabeçalho da tabela.

```
<thead><tr><th>Nome</th></tr></thead>
```

#38 <tbody> — TABELA

Agrupa as linhas do corpo (os dados) da tabela.

```
<tbody><tr><td>Ana</td></tr></tbody>
```

#39 <h4> — TEXTO

Título de 4º nível. Siga a hierarquia: não pule níveis!

```
<h4>Ingredientes</h4>
```

#40 <h5> — TEXTO

Título de 5º nível, para conteúdos bem aninhados.

```
<h5>Observação</h5>
```

#41 <h6> — TEXTO

Título de 6º e último nível, o menor de todos.

```
<h6>Nota</h6>
```

#42 — TEXTO

Ênfase na fala (muda o sentido da frase). Exibido em itálico.

```
Eu <em>adoro</em> pizza!
```

#43 <b> — TEXTO

Negrito só visual, sem importância extra (palavras-chave, nomes de produtos).

```
<b>Promoção</b> de verão
```

#44 <select> — FORMULÁRIO

Caixa de seleção (dropdown). As opções ficam em tags `<option>`.

```
<select name="uf">
  <option>SP</option>
</select>
```

#45 <option> — FORMULÁRIO

Uma opção do `<select>` ou `<datalist>`. O `value` é o que vai para o servidor.

```
<option value="rj">Rio de
Janeiro</option>
```

#46 <textarea> — FORMULÁRIO

Campo de texto com várias linhas (comentários, mensagens).

```
<textarea name="msg" rows="4">
</textarea>
```

#47 <iframe> — MÍDIA

Incorpora outra página dentro da sua: vídeos do YouTube, mapas, etc.

```
<iframe
src="https://youtube.com/embed/ID">
</iframe>
```

#48 <svg> — MÍDIA

Desenho vetorial direto no código: ícones e ilustrações que não perdem qualidade.

```
<svg width="20" height="20">
  <circle cx="10" cy="10" r="8"/>
</svg>
```

#49 <aside> — SEMÂNTICA

Conteúdo lateral, relacionado mas não essencial: barra lateral, dicas, anúncios.

```
<aside>Posts populares</aside>
```

#50 <hr> — TEXTO

Linha horizontal que marca mudança de assunto. Não tem fechamento.

```
<p>Parte 1</p><hr><p>Parte 2</p>
```

#51 <figure> — MÍDIA

Agrupa uma mídia (imagem, gráfico, código) com a sua legenda.

```
<figure><img src="g.png" alt="...">
</figure>
```

Dica do Esqueleto: Sempre preencha o `alt` das imagens: acessibilidade em primeiro lugar.

Tags HTML — da mais usada à menos usada

#52 <figcaption> — MÍDIA

Legenda de um `<figure>`.

```
<figcaption>Fig. 1 – Vendas</figcaption>
```

#53 <video> — MÍDIA

Reproduz vídeo. Com `controls` aparecem play, volume e tela cheia.

```
<video src="aula.mp4" controls></video>
```

#54 <audio> — MÍDIA

Reproduz áudio (músicas, podcasts, efeitos).

```
<audio src="som.mp3" controls></audio>
```

#55 <source> — MÍDIA

Fontes alternativas de mídia para `video`, `audio` ou `picture`. O navegador usa a primeira compatível.

```
<source src="v.webm" type="video/webm">
```

#56 <picture> — MÍDIA

Entrega imagens diferentes conforme a tela ou o formato suportado.

```
<picture>
  <source srcset="a.webp"><img
src="a.jpg" alt="">
</picture>
```

#57 <small> — TEXTO

Texto de menor importância: letras miúdas, avisos legais.

```
<small>*Imagens ilustrativas</small>
```

#58 <blockquote> — TEXTO

Citação longa, em bloco, geralmente com recuo.

```
<blockquote>Seja a mudança...
</blockquote>
```

#59 <code> — TEXTO

Trecho de código dentro de um texto (fonte monoespaçada).

```
Use <code>console.log()</code>
```

#60 <pre> — TEXTO

Texto pré-formatado: respeita espaços e quebras de linha como foram digitados.

```
<pre><code>let x = 1;</code></pre>
```

#61 <noscript> — SCRIPT

Conteúdo exibido só se o JavaScript estiver desativado.

```
<noscript>Ative o JavaScript!</noscript>
```

#62 <fieldset> — FORMULÁRIO

Agrupa campos relacionados de um formulário, com uma borda ao redor.

```
<fieldset>...campos...</fieldset>
```

#63 <legend> — FORMULÁRIO

Título de um `<fieldset>`, aparece "cortando" a borda.

```
<legend>Dados pessoais</legend>
```

#64 <dl> — LISTA

Lista de descrição: pares de termo e definição (glossários, FAQs).

```
<dl><dt>HTML</dt><dd>Estrutura</dd></dl>
```

#65 <dt> — LISTA

Termo dentro de uma lista de descrição `<dl>`.

```
<dt>CSS</dt>
```

#66 <dd> — LISTA

Definição do termo `<dt>` anterior.

```
<dd>Linguagem de estilo</dd>
```

#67 <canvas> — MÍDIA

Área de desenho controlada por JavaScript: gráficos, jogos, animações.

```
<canvas id="jogo" width="300"
height="150"></canvas>
```

#68 <time> — SEMÂNTICA

Data/hora legível por máquinas pelo atributo `datetime`.

```
<time datetime="2026-12-25">Natal</time>
```

#69 <address> — SEMÂNTICA

Informações de contato do autor ou da empresa.

```
<address>contato@site.com</address>
```

#70 <details> — INTERATIVO

Bloco que abre e fecha (sanfona/acordeão) sem nenhum JavaScript.

```
<details>
  <summary>Ver mais</summary>Texto
</details>
```

#71 <summary> — INTERATIVO

O título clicável de um `<details>`.

```
<summary>Perguntas frequentes</summary>
```

#72 <dialog> — INTERATIVO

Caixa de diálogo (modal) nativa. Abre com `showModal()` no JavaScript.

```
<dialog id="modal">Olá!</dialog>
```

#73 <u> — TEXTO

Sublinha o texto (ex.: erro de ortografia). Cuidado para não confundir com link.

```
<u>Texto sublinhado</u>
```

#74 <s> — TEXTO

Texto riscado: algo que não é mais válido.

```
<s>R$ 99</s> R$ 49
```

#75 <mark> — TEXTO

Destaca o texto como um marca-texto (fundo amarelo).

```
Achei a <mark>palavra</mark>!
```

#76 <sup> — TEXTO

Sobrescrito: texto menor e mais alto (expoentes, 1º).

```
x<sup>2</sup>
```

#77 <sub> — TEXTO

Subscrito: texto menor e mais baixo (fórmulas químicas).

```
H<sub>2</sub>O
```

#78 <abbr> — TEXTO

Abreviação ou sigla. O `title` mostra o significado ao passar o mouse.

```
<abbr title="HyperText Markup
Language">HTML</abbr>
```

Dica do Esqueleto: Prefira tags semânticas (`header` , `nav` , `main` , `footer`) a uma "sopa" de `div` .

Tags HTML — da mais usada à menos usada

#79 <cite>
TEXTO

Título de uma obra (livro, filme, música).

```
<cite>Dom Casmurro</cite>
```

#80 <q>
TEXTO

Citação curta na linha. O navegador coloca as aspas sozinho.

```
Ele disse <q>volto já</q>.
```

#81 <kbd>
TEXTO

Representa teclas e atalhos do teclado.

```
Aperte <kbd>Ctrl</kbd> + <kbd>C</kbd>
```

#82 <tfoot>
TABELA

Agrupa as linhas de rodapé da tabela (totais, resumos).

```
<tfoot><tr><td>Total</td></tr></tfoot>
```

#83 <caption>
TABELA

Título da tabela. Deve ser o primeiro filho de `<table>`.

```
<caption>Notas do bimestre</caption>
```

#84 <colgroup>
TABELA

Agrupa colunas da tabela para estilizá-las de uma vez.

```
<colgroup><col span="2"></colgroup>
```

#85 <col>
TABELA

Uma coluna dentro de `<colgroup>`. Não tem fechamento.

```
<col style="background:#eee">
```

#86 <optgroup>
FORMULÁRIO

Agrupa opções de um `<select>` sob um rótulo.

```
<optgroup label="Frutas">...</optgroup>
```

#87 <datalist>
FORMULÁRIO

Lista de sugestões (autocompletar) para um `<input>` ligado pelo atributo `list`.

```
<input list="cores">
<datalist id="cores">...</datalist>
```

#88 <output>
FORMULÁRIO

Mostra o resultado de um cálculo ou ação do usuário.

```
<output name="total">0</output>
```

#89 <progress>
FORMULÁRIO

Barra de progresso de uma tarefa (download, etapas).

```
<progress value="70" max="100">
</progress>
```

#90 <meter>
FORMULÁRIO

Medidor de um valor em uma faixa conhecida (bateria, nota, disco).

```
<meter value="0.6" min="0" max="1">
</meter>
```

#91 <template>
SCRIPT

Molde de HTML invisível que o JavaScript clona para criar elementos repetidos.

```
<template id="item"><li></li></template>
```

#92 <slot>
SCRIPT

Ponto de encaixe de conteúdo em Web Components (Shadow DOM).

```
<slot name="titulo"></slot>
```

#93 <track>
MÍDIA

Legendas sincronizadas para vídeo e áudio (arquivos .vtt).

```
<track src="pt.vtt" kind="subtitles"
srclang="pt">
```

#94 <embed>
MÍDIA

Incorpora conteúdo externo (como um PDF). Não tem fechamento.

```
<embed src="manual.pdf"
type="application/pdf">
```

#95 <object>
MÍDIA

Incorpora um recurso externo, com conteúdo alternativo se falhar.

```
<object data="manual.pdf"></object>
```

#96 <map>
MÍDIA

Mapa de imagem: define áreas clicáveis em uma imagem (com `usemap`).

```
<map name="mapa">...</map>
```

#97 <area>
MÍDIA

Área clicável dentro de um `<map>` (retângulo, círculo ou polígono).

```
<area shape="circle" coords="50,50,20"
href="sp.html" alt="SP">
```

#98 <base>
METADADOS

Define o endereço base de todos os links relativos. Só pode haver um.

```
<base href="https://site.com/">
```

#99 <wbr>
TEXTO

Sugere onde uma palavra muito longa pode quebrar de linha.

```
super<wbr>cali<wbr>fragilístico
```

#100 <bdi>
TEXTO

Isola um texto de direção desconhecida (ex.: nomes em árabe).

```
Usuário: <bdi>إيان</bdi>
```

#101 <bdo>
TEXTO

Força a direção do texto com o atributo `dir`.

```
<bdo dir="rtl">Olá</bdo>
```

#102 <ruby>
TEXTO

Anotação de pronúncia acima de caracteres (idiomas asiáticos), junto com `rt` e `rp`.

```
<ruby>漢<rt>kan</rt></ruby>
```

#103 <rt>
TEXTO

O texto de pronúncia dentro do `<ruby>`.

```
<rt>kan</rt>
```

#104 <rp>
TEXTO

Parênteses para navegadores sem suporte a `<ruby>`.

```
<rp>(</rp><rt>kan</rt><rp>)</rp>
```

#105 <data>
SEMÂNTICA

Liga um valor legível por máquina a um conteúdo.

```
<data value="7891">Café 500g</data>
```

Dica do Esqueleto: Comentário em HTML: `<!-- isto não aparece na tela -->`

Tags HTML — da mais usada à menos usada

#106 <var>

TEXTO

Variável em expressões matemáticas ou de programação.

```
<var>x</var> + 2 = 5
```

#107 <samp>

TEXTO

Exemplo de saída (resposta) de um programa.

```
<samp>Arquivo não encontrado</samp>
```

#108 <dfn>

TEXTO

Marca o termo que está sendo definido naquele trecho.

```
<dfn>HTML</dfn> é uma linguagem...
```

#109 <ins>

TEXTO

Texto inserido em uma revisão (aparece sublinhado).

```
<ins>novo trecho</ins>
```

#110 <del>

TEXTO

Texto removido em uma revisão (aparece riscado).

```
<del>trecho antigo</del>
```

#111 <hgroup>

SEMÂNTICA

Agrupa um título com o seu subtítulo.

```
<hgroup><h1>Título</h1>
<p>Sub</p></hgroup>
```

#112 <search>

SEMÂNTICA

Envolve uma área de busca ou filtros (tag recente do HTML).

```
<search><input
type="search"></search>
```

#113 <menu>

LISTA

Lista de comandos/ações, como uma `<ul>` para barras de ferramentas.

```
<menu><li>
<button>Copiar</button>
</li></menu>
```

Dica do Esqueleto: O `id` deve ser único na página; a `class` pode se repetir à vontade.

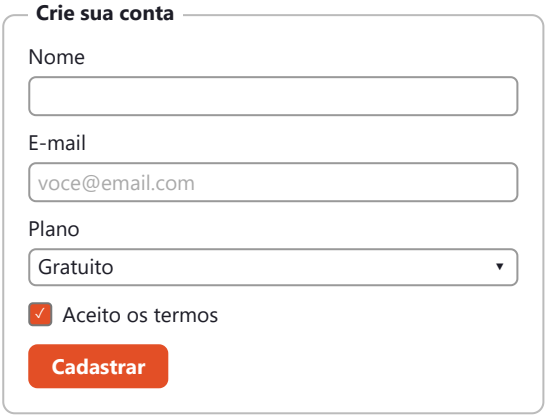
Formulários em ação

Formulários são a principal forma de o usuário **enviar dados**: cadastros, logins, buscas, comentários. A tag `<form>` envolve os campos, e cada campo com `name` é enviado como um par `nome=valor`.

CADASTRO.HTML

```
<form action="/cadastro" method="post">  1
  <fieldset>
    <legend>Crie sua conta</legend>  2
    <label for="nome">Nome</label>  3
    <input id="nome" name="nome" required>
    <label for="email">E-mail</label>
    <input type="email" id="email" name="email">
    <label for="plano">Plano</label>
    <select id="plano" name="plano">  4
      <option value="free">Gratuito</option>
      <option value="pro">Pro</option>
    </select>
    <label>
      <input type="checkbox" name="termos">  5
      Aceito os termos
    </label>
    <button type="submit">Cadastrar</button>  6
  </fieldset>
</form>
```

COMO FICA NA TELA



- **1** `action` = para onde vai; `method` = como vai (`post` esconde os dados da URL).
- **2** `fieldset` + `legend` agrupam e dão título.
- **3** `label for` ↔ `input id`: clicar no texto foca o campo.
- **4** `select` cria uma lista de opções.
- **5** Checkbox dentro do label: o texto todo fica clicável.
- **6** O botão `submit` envia: `nome=Ana&email=ana@x.com&plano=free`

Todos os tipos de <input type="...">

text Texto de uma linha (padrão).	**email** Valida se tem formato de e-mail.	**password** Esconde o que é digitado (••••).	**number** Só números, com setinhas.
tel Telefone (teclado numérico no celular).	**url** Valida endereços web.	**search** Campo de busca (com botão de limpar).	**date** Abre um calendário.
time Escolhe um horário.	**datetime-local** Data e hora juntas.	**month · week** Escolhe mês ou semana.	**color** Abre um seletor de cores.
range Controle deslizante (slider).	**file** Envio de arquivos (upload).	**checkbox** Caixinha: marca várias opções.	**radio** Bolinha: escolhe UMA do grupo (mesmo `name`).
hidden Invisível: envia dados escondidos.	**submit** Botão que envia o formulário.	**reset** Botão que limpa os campos.	**button** Botão sem ação padrão (para JS).

GET ou POST?

	GET	POST
Dados vão	Na URL: `?busca=tenis`	No corpo da requisição
Use para	Buscas e filtros	Cadastros, logins, senhas
Favoritar?	Sim, a URL guarda tudo	Não

Validação sem nenhum JavaScript

```
<input type="email" required>
<input minlength="8" type="password">
<input pattern="[0-9]{5}-[0-9]{3}"
       placeholder="00000-000">
```

O navegador bloqueia o envio e mostra a mensagem de erro sozinho.

Dica do Esqueleto: Aperte `Ctrl + U` no navegador para ver o HTML de qualquer site.

Atributos — dos principais aos menos usados

Por que existem atributos?

Se as tags são os ossos do esqueleto, os **atributos** são as **configurações** presas a eles. Uma tag sozinha diz *o que* o elemento é; o atributo diz *como* ele deve funcionar. Um link sem `href` não sabe para onde ir; uma imagem sem `src` não sabe o que mostrar.

Eles ficam sempre **dentro da tag de abertura**, no formato `nome="valor"`, separados por espaço. Servem para: **identificar** (`id`, `class`), **apontar caminhos** (`href`, `src`), **descrever** (`alt`, `title`, `lang`), **configurar comportamento** (`type`, `required`, `disabled`), **ligar o HTML ao CSS e ao JS** (`class`, `id`, `data-*`) e garantir **acessibilidade** (`alt`, `aria-*`, `role`).

```
<a href="site.com">
tag  atributo  valor
```

GLOBAIS
Funcionam em **qualquer** tag: `id`, `class`, `style`, `title`, `hidden`, `data-*`...

ESPECÍFICOS
Só fazem sentido em algumas tags: `href` no `a`, `src` no `img`, `action` no `form`.

BOOLEANOS
Não precisam de valor: só de existir já valem "sim". Ex.: `required`, `disabled`, `checked`.

#01 class — GLOBAL

Dá um ou mais "apelidos" ao elemento para estilizar no CSS (`.nome`) ou selecionar no JS. Pode se repetir.

```
<div class="card destaque">
```

#02 id — GLOBAL

Identificador ÚNICO na página. Usado em âncoras (`#id`), no `for` do label e no JS.

```
<section id="contato">
```

#03 href — A, LINK, BASE

Endereço de destino de um link ou do recurso conectado.

```
<a href="#contato">Contato</a>
```

#04 src — IMG, SCRIPT, VIDEO, IFRAME

Caminho do arquivo que será carregado.

```
<img src="img/logo.png" alt="Logo">
```

#05 alt — IMG, AREA, INPUT

Texto alternativo da imagem, lido por leitores de tela. Essencial para acessibilidade!

```
<img src="pao.jpg" alt="Pão francês">
```

#06 style — GLOBAL

Aplica CSS direto no elemento (inline). Tem prioridade alta: use com moderação.

```
<p style="color: blue;">Azul</p>
```

#07 type — INPUT, BUTTON, SCRIPT

Tipo do elemento: no `input` muda o campo; no `button` o comportamento (`submit`, `button`, `reset`).

```
<input type="password">
```

#08 name — INPUT, SELECT, FORM

Nome do campo enviado ao servidor junto com o valor (`nome=valor`).

```
<input name="email">
```

#09 value — INPUT, OPTION, BUTTON

Valor do campo ou da opção. No input, é o valor inicial.

```
<input value="Paulo">
```

#10 placeholder — INPUT, TEXTAREA

Texto de dica que aparece dentro do campo vazio.

```
<input placeholder="Digite seu nome">
```

#11 rel — A, LINK

Relação com o recurso: `stylesheet`, `icon`, `noopener`, `nofollow`...

```
<link rel="icon" href="favicon.png">
```

#12 target — A, FORM

Onde abrir o link: `_blank` abre em nova aba; `_self` (padrão) na mesma.

```
<a href="..." target="_blank">
```

#13 title — GLOBAL

Dica (tooltip) que aparece ao passar o mouse.

```
<button title="Salvar arquivo">
```

#14 width · height — IMG, VIDEO, CANVAS

Largura e altura. Em imagens, evitam que a página "pule" ao carregar.

```
<img src="a.png" width="300"
height="200">
```

#15 charset — META

Codificação de caracteres. `UTF-8` garante acentos e emojis corretos.

```
<meta charset="UTF-8">
```

#16 content — META

O valor da informação do `<meta>` (descrição, viewport...).

```
<meta name="description"
content="Bolos">
```

#17 lang — GLOBAL

Idioma do conteúdo. Ajuda tradutores, leitores de tela e o Google.

```
<html lang="pt-BR">
```

#18 for — LABEL, OUTPUT

Liga o label ao campo que tem o `id` correspondente.

```
<label for="email">E-mail</label>
```

Dica do Esqueleto: Use `<button>` para ações e `<a>` para navegar entre páginas.

Atributos — dos principais aos menos usados

#19 action
FORM

Endereço (URL) para onde os dados do formulário vão.

```
<form action="/login">
```

#20 method
FORM

Como enviar: `get` (dados na URL, para buscas) ou `post` (no corpo, para cadastros e senhas).

```
<form method="post">
```

#21 required
INPUT, SELECT, TEXTAREA

Campo obrigatório: o navegador bloqueia o envio se estiver vazio.

```
<input required>
```

#22 disabled
INPUT, BUTTON, SELECT

Desativa o elemento: não clica, não edita e não é enviado.

```
<button disabled>Aguarde</button>
```

#23 checked
INPUT CHECKBOX/RADIO

Deixa a caixinha ou opção já marcada.

```
<input type="checkbox" checked>
```

#24 selected
OPTION

Deixa a opção já escolhida no `<select>`.

```
<option selected>Brasil</option>
```

#25 readonly
INPUT, TEXTAREA

Somente leitura: é enviado, mas não pode ser editado.

```
<input value="123" readonly>
```

#26 data-*
GLOBAL

Atributos personalizados para guardar dados extras, lidos no JS por `dataset`.

```
<button data-id="42">Comprar</button>
```

#27 aria-label
GLOBAL · ACESSIBILIDADE

Nome acessível para leitores de tela quando não há texto visível (botões só com ícone).

```
<button aria-label="Fechar">X</button>
```

#28 role
GLOBAL · ACESSIBILIDADE

Informa a função do elemento para tecnologias assistivas.

```
<div role="alert">Erro!</div>
```

#29 tabindex
GLOBAL

Ordem de foco pelo Tab. `0` torna focável; `-1` foca só via JS.

```
<div tabindex="0">
```

#30 hidden
GLOBAL

Esconde o elemento como se ele não existisse na tela.

```
<p hidden>Segredo</p>
```

#31 onclick · on*
GLOBAL · EVENTOS

Roda JS quando o evento acontece (`onclick`, `onchange`...). Prefira `addEventListener`.

```
<button onclick="alert('Oi')">
```

#32 loading
IMG, IFRAME

`lazy` só carrega quando a imagem se aproxima da tela.

```
<img src="f.jpg" loading="lazy"
alt="">
```

#33 srcset
IMG, SOURCE

Versões da imagem em vários tamanhos; o navegador escolhe a ideal.

```
srcset="p.jpg 480w, g.jpg 1080w"
```

#34 sizes
IMG, SOURCE

Diz qual largura a imagem ocupará, para escolher do `srcset`.

```
sizes="(max-width: 600px) 100vw, 50vw"
```

#35 autocomplete
INPUT, FORM

Liga/desliga o preenchimento automático e dá dicas (`email`, `name`).

```
<input autocomplete="email">
```

#36 autofocus
INPUT, BUTTON

Coloca o foco no elemento assim que a página abre.

```
<input autofocus>
```

#37 maxlength · minlength
INPUT, TEXTAREA

Quantidade máxima e mínima de caracteres.

```
<input maxlength="11" minlength="11">
```

#38 min · max
INPUT NUMBER/DATE/RANGE

Menor e maior valor aceito.

```
<input type="number" min="1" max="10">
```

#39 step
INPUT NUMBER/RANGE

Intervalo entre os valores permitidos.

```
<input type="number" step="0.5">
```

#40 pattern
INPUT

Expressão regular que o valor precisa obedecer.

```
<input pattern="[0-9]{5}-[0-9]{3}">
```

#41 multiple
INPUT, SELECT

Permite escolher mais de um valor ou arquivo.

```
<input type="file" multiple>
```

#42 accept
INPUT FILE

Tipos de arquivo aceitos no upload.

```
<input type="file"
accept="image/*,.pdf">
```

#43 enctype
FORM

Codificação do envio. Use `multipart/form-data` para mandar arquivos.

```
<form enctype="multipart/form-data">
```

#44 colspan · rowspan
TD, TH

Faz a célula ocupar várias colunas ou linhas.

```
<td colspan="2">Total</td>
```

#45 defer
SCRIPT

Baixa em paralelo e só executa quando o HTML está pronto, na ordem.

```
<script src="app.js" defer></script>
```

Dica do Esqueleto: Todo `input` merece um `label` ligado pelo par `for` / `id`.

Atributos — dos principais aos menos usados

#46 async
SCRIPT

Baixa em paralelo e executa assim que termina (sem ordem). Bom para analytics.

```
<script src="ga.js" async></script>
```

#47 download
A

Faz o link baixar o arquivo em vez de abrir.

```
<a href="cv.pdf" download>Baixar</a>
```

#48 controls
VIDEO, AUDIO

Mostra os controles de reprodução.

```
<video src="v.mp4" controls>
```

#49 autoplay
VIDEO, AUDIO

Começa a tocar sozinho (quase sempre exige `muted`).

```
<video autoplay muted>
```

#50 loop
VIDEO, AUDIO

Repete a mídia sem parar.

```
<audio src="fundo.mp3" loop>
```

#51 muted
VIDEO, AUDIO

Começa sem som.

```
<video muted>
```

#52 poster
VIDEO

Imagem de capa mostrada antes do vídeo tocar.

```
<video poster="capa.jpg">
```

#53 contenteditable
GLOBAL

Deixa o conteúdo do elemento editável pelo usuário.

```
<div contenteditable="true">
```

#54 draggable
GLOBAL

Permite (ou não) arrastar o elemento.

```
<img draggable="false">
```

#55 spellcheck
GLOBAL

Liga/desliga a correção ortográfica.

```
<textarea spellcheck="false">
```

#56 dir
GLOBAL

Direção do texto: `ltr`, `rtl` ou `auto`.

```
<p dir="rtl">
```

#57 rows · cols
TEXTAREA

Linhas visíveis e largura em caracteres.

```
<textarea rows="5" cols="40">
```

#58 label
OPTGROUP, OPTION, TRACK

Rótulo de um grupo de opções ou de uma legenda.

```
<optgroup label="Bebidas">
```

#59 list
INPUT

Liga o input a um `<datalist>` pelo id.

```
<input list="cidades">
```

#60 form
INPUT, BUTTON, SELECT

Liga o campo a um formulário pelo id, mesmo fora dele.

```
<button form="cadastro">Enviar</button>
```

#61 novalidate
FORM

Desliga a validação automática do navegador.

```
<form novalidate>
```

#62 open
DETAILS, DIALOG

Deixa o `<details>` ou `<dialog>` aberto.

```
<details open>
```

#63 datetime
TIME, INS, DEL

Data/hora em formato de máquina (AAAA-MM-DD).

```
<time datetime="2026-10-14">
```

#64 cite
BLOCKQUOTE, Q

URL da fonte de uma citação.

```
<blockquote cite="https://...">
```

#65 sandbox
IFRAME

Aplica restrições de segurança ao iframe.

```
<iframe sandbox="allow-scripts">
```

#66 allow
IFRAME

Permissões do iframe (tela cheia, câmera, autoplay).

```
<iframe allow="fullscreen">
```

#67 http-equiv
META

Simula um cabeçalho HTTP (ex.: recarregar a página).

```
<meta http-equiv="refresh" content="30">
```

#68 media
LINK, SOURCE, STYLE

Só aplica o recurso se a condição de mídia for verdadeira.

```
<link href="print.css" media="print">
```

#69 scope
TH

Diz se o cabeçalho é da coluna (`col`) ou da linha (`row`).

```
<th scope="col">Nome</th>
```

#70 headers
TD, TH

Liga a célula aos ids dos cabeçalhos (tabelas complexas).

```
<td headers="nome">
```

#71 span
COL, COLGROUP

Quantas colunas o `<col>` abrange.

```
<col span="3">
```

#72 start · reversed
OL

Número inicial da lista / contagem de trás pra frente.

```
<ol start="5" reversed>
```

#73 referrerpolicy
A, IMG, IFRAME

Controla quais dados de origem vão na requisição.

```
referrerpolicy="no-referrer"
```

#74 translate
GLOBAL

`no` impede que tradutores automáticos traduzam o trecho.

```
<span translate="no">Work in Tech</span>
```

#75 inputmode
CAMPOS

Qual teclado mostrar no celular (`numeric`, `email`, `tel`).

```
<input inputmode="numeric">
```

Dica do Esqueleto: HTML não é linguagem de programação: é linguagem de MARCAÇÃO. Ele estrutura, não "pensa".

Atributos — dos principais aos menos usados

#76 enterkeyhint

CAMPOS

Muda o rótulo da tecla Enter no celular (`search`, `send`).

```
<input
enterkeyhint="send">
```

#77 popover

GLOBAL

Transforma o elemento em popover nativo, aberto por `popovertarget`.

```
<div id="dica"
popover>Oi!</div>
```

#78 inert

GLOBAL

Torna o elemento e os filhos não interativos.

```
<main inert>
```

#79 accesskey

GLOBAL

Atalho de teclado para ativar o elemento.

```
<button accesskey="s">
```

#80 decoding

IMG

Dica de decodificação da imagem: `async`, `sync`, `auto`.

```
<img decoding="async">
```

#81 fetchpriority

IMG, LINK, SCRIPT

Prioridade de download: `high`, `low` ou `auto`.

```
<img
fetchpriority="high">
```

#82 integrity · crossorigin

SCRIPT, LINK

Confere se o arquivo do CDN não foi alterado e define regras entre domínios.

```
integrity="sha384-..."
crossorigin="anonymous"
```

#83 hreflang

A, LINK

Idioma da página de destino do link.

```
<link rel="alternate"
hreflang="en">
```

#84 nonce

SCRIPT, STYLE

Código único usado pela política de segurança (CSP).

```
<script nonce="abc123">
```

Dica do Esqueleto: No VS Code, digite `!` e aperte `Tab` para gerar a estrutura básica.



CAPÍTULO 2

CSS

A pele e os músculos da página

CSS (*Cascading Style Sheets* — Folhas de Estilo em Cascata) é quem dá **aparência** ao HTML. Se o HTML é o esqueleto, o CSS é a **pele, os músculos, as roupas e o cabelo**: cores, fontes, tamanhos, espaçamentos, posições, sombras e animações.

Mas repare nos nossos amigos aqui embaixo: eles são lindos, mas **não têm cérebro**. O CSS deixa tudo bonito, porém não toma decisões — quem pensa é o JavaScript.

```
h1 { color: #E34F26; }
```

seletor · abre · propriedade · valor · fim · fecha

propriedade: valor; = uma **declaração**. Seletor + bloco de declarações = uma **REGRA** CSS.

Por que "cascata"?

Várias regras podem mirar o mesmo elemento. Elas "escorrem" umas sobre as outras e o navegador decide qual vence.

Um arquivo, mil páginas

Um único `style.css` pode vestir o site inteiro. Mudou ali, mudou em todas as páginas.

Seletor = mira

O seletor escolhe **quem** recebe o estilo; as declarações dizem **como** ele vai ficar.

Neste capítulo

Como aplicar o CSS e como a cascata funciona 17

Box Model, unidades de medida e especificidade 18

Seletores CSS: do mais usado ao mais avançado 19

Propriedades CSS em 8 níveis de dificuldade 20

Cores em hexadecimal (página especial) 25

Como aplicar o CSS: as 3 formas e a cascata

Existem três jeitos de "vestir" o HTML com CSS. Todos funcionam, mas só um é o recomendado para projetos de verdade.

1 · Inline evite

```html
<p style="color: red;">
  Texto vermelho
</p>
```

Direto no atributo `style`. Difícil de manter e repete código.

2 · Interno ok p/ testes

```html
<head>
  <style>
    p { color: red; }
  </style>
</head>
```

Na tag `<style>`. Vale só para aquela página.

3 · Externo recomendado

```html
<head>
  <link rel="stylesheet"
        href="style.css">
</head>
```

Arquivo `.css` separado, ligado pelo `<link>`. Organizado e reutilizável!

A cascata: quem vence a briga?

Quando duas regras mandam no mesmo elemento, o navegador decide nesta ordem:

- **Importância:** `!important` passa na frente de tudo (use quase nunca).
- **Especificidade:** o seletor mais "específico" vence (`#id` > `.classe` > `tag`). Veja a próxima página.
- **Ordem:** empatou? Vence a regra escrita **por último**.

STYLE.CSS

```css
p { color: blue; }
p { color: green; } /* vence: veio depois */

.aviso { color: orange; }
p { color: purple; } /* perde: classe é mais específica */
```

Herança: o filho puxa o pai

Algumas propriedades passam automaticamente do pai para os filhos — principalmente as de **texto**: `color`, `font-family`, `font-size`, `line-height`. Já as de **caixa** (`margin`, `padding`, `border`, `background`) **não** são herdadas.

```css
body { font-family: Arial; color: #333; }
/* todos os textos da página herdam isso! */
```

Organizando o arquivo

```css
/* ===== 1. Variáveis ===== */
:root { --azul: #1B3A8C; }

/* ===== 2. Reset ===== */
* { margin: 0; box-sizing: border-box; }

/* ===== 3. Componentes ===== */
.botao { background: var(--azul); }
```

Os 3 seletores do dia a dia

```css
p { color: #333; }          /* tag     */
.destaque { color: #E34F26; } /* classe */
#topo { height: 80px; }       /* id      */
```

Tag (sem símbolo), classe (com **ponto**) e id (com **cerquilha #**). Com eles você já estiliza 90% de um site. A lista completa vem nas próximas páginas!

Checklist do CSS caprichado

- Um arquivo `style.css` externo, ligado no `<head>`.
- `box-sizing: border-box` em tudo.
- Cores e fontes em **variáveis** no `:root`.
- Classes com nomes claros: `.card-produto`, `.btn-primario`.
- Layout com **Flexbox** e **Grid** (nada de tabelas!).
- Testar no celular com `@media`.

Experimente agora: abra qualquer site, aperte F12, clique em um elemento na aba *Elements* e mude os valores do painel *Styles*. Nada quebra de verdade — é só recarregar a página!

Dica do Músculo: Aperte `F12` e use o inspetor para testar CSS ao vivo, sem medo.

Box Model, unidades e especificidade

Box Model: todo elemento é uma caixa



▲ a faixa azul-escura é a **border** (borda)

```css
.caixa {
  width: 200px;
  padding: 20px;
  border: 5px solid #1B3A8C;
  margin: 30px;
}
```

Conta da largura real: 200 + 20 + 20 + 5 + 5 = **250px**. Com `box-sizing: border-box`, a caixa fica com **200px** no total (o padding e a borda entram "para dentro"). Por isso todo mundo usa!

Valores em atalho: `margin: 10px` (todos) · `10px 20px` (vertical horizontal) · `1px 2px 3px 4px` (topo, direita, baixo, esquerda — sentido do relógio).

Unidades de medida

Unidade	Tipo	Significa	Quando usar
`px`	Absoluta	Pixels da tela. Sempre o mesmo tamanho.	Bordas, sombras, detalhes
`%`	Relativa ao pai	Porcentagem do tamanho do elemento pai.	Larguras fluidas
`rem`	Relativa à raiz	Múltiplo da fonte do `html` (normalmente 16px). `2rem` = 32px.	Fontes e espaçamentos
`em`	Relativa ao elemento	Múltiplo da fonte do próprio elemento (ou do pai).	Padding de botões
`vw · vh`	Relativa à tela	1% da largura / altura da janela. `100vh` = tela toda.	Seções de tela cheia
`fr`	Fração (Grid)	Uma fração do espaço livre da grade.	Colunas do Grid
`ch`	Relativa	Largura do caractere "0" da fonte.	Limitar linhas de texto
`deg · s · ms`	Ângulo · tempo	Graus de rotação · segundos e milissegundos.	Transform e animações

Especificidade: o placar dos seletores

1-0-0-0	0-1-0-0	0-0-1-0	0-0-0-1
style="" inline	#id	.classe · :hover · [attr]	tag · ::before

Compare da esquerda para a direita, como um placar: uma única `#id` vence qualquer quantidade de classes. O seletor `*` vale 0.

Seletor	Placar
`p`	0·0·0·1
`nav a`	0·0·0·2
`.menu a:hover`	0·0·2·1
`#topo .logo`	0·1·1·0 **vence**

Dica do Músculo: Comece todo projeto com `* { box-sizing: border-box; }`

Seletores — escolhendo quem recebe o estilo

#01 * — UNIVERSAL

Seleciona TODOS os elementos. Muito usado para zerar margens.

```
* { margin: 0; box-sizing: border-box; }
```

#02 tag — TIPO

Seleciona todos os elementos daquela tag.

```
p { color: #333; }
```

#03 .classe — CLASSE

Seleciona os elementos com a classe. O mais usado no dia a dia!

```
.botao { background: blue; }
```

#04 #id — ID

Seleciona o elemento com aquele id (único). Especificidade alta.

```
#topo { height: 80px; }
```

#05 A, B — AGRUPAMENTO

Aplica a mesma regra a vários seletores de uma vez.

```
h1, h2, h3 { font-family: Arial; }
```

#06 A B — DESCENDENTE

Seleciona B em qualquer nível dentro de A.

```
nav a { text-decoration: none; }
```

#07 A > B — FILHO DIRETO

Seleciona B só se for filho direto de A.

```
ul > li { list-style: none; }
```

#08 :hover — PSEUDO-CLASSE

Quando o mouse está em cima do elemento.

```
.btn:hover { opacity: .8; }
```

#09 :focus — PSEUDO-CLASSE

Quando o elemento está focado (clique ou Tab).

```
input:focus { border-color: blue; }
```

#10 :active — PSEUDO-CLASSE

No instante em que o elemento está sendo clicado.

```
.btn:active { transform: scale(.95); }
```

#11 A + B — IRMÃO ADJACENTE

O B que vem IMEDIATAMENTE depois de A.

```
h2 + p { margin-top: 0; }
```

#12 A ~ B — IRMÃOS GERAIS

Todos os B que vêm depois de A (mesmo pai).

```
h2 ~ p { color: gray; }
```

#13 [atributo] — ATRIBUTO

Seleciona por atributo ou pelo valor dele.

```
input[type="email"] { width: 100%; }
```

#14 :first-child · :last-child — PSEUDO-CLASSE

Primeiro ou último filho dentro do pai.

```
li:last-child { border: none; }
```

#15 :nth-child(n) — PSEUDO-CLASSE

Pela posição: número, `odd`, `even` ou fórmula `2n+1`.

```
tr:nth-child(even) { background: #eee; }
```

#16 ::before · ::after — PSEUDO-ELEMENTO

Cria um "filho" virtual antes/depois do conteúdo. Exige `content`.

```
.novo::after { content: " ★"; }
```

#17 ::placeholder — PSEUDO-ELEMENTO

Estiliza o texto de dica dos campos.

```
::placeholder { color: #aaa; }
```

#18 :not() — PSEUDO-CLASSE

Tudo que NÃO corresponde ao seletor.

```
li:not(.ativo) { opacity: .5; }
```

#19 :root — PSEUDO-CLASSE

O elemento raiz (`<html>`). Lugar ideal para variáveis.

```
:root { --laranja: #E34F26; }
```

#20 :checked · :disabled — PSEUDO-CLASSE

Campos marcados / campos desativados.

```
input:checked + label { font-weight: bold; }
```

#21 ::selection — PSEUDO-ELEMENTO

O texto que o usuário seleciona com o mouse.

```
::selection { background: yellow; }
```

#22 :focus-visible — PSEUDO-CLASSE

Foco visível só quando faz sentido (teclado).

```
a:focus-visible { outline: 2px solid; }
```

#23 :is() · :where() — PSEUDO-CLASSE

Agrupa seletores de forma curta. `:where()` tem especificidade zero.

```
:is(h1, h2) span { color: red; }
```

#24 :has() — PSEUDO-CLASSE

O "seletor pai": seleciona quem CONTÉM algo.

```
.card:has(img) { padding: 0; }
```

Dica do Músculo: Prefira classes a ids para estilizar: classes são reutilizáveis.

Propriedades — do básico ao mais complexo

Nível 1 · Texto e cores — o básico do básico

#01 color — TEXTO

Cor do texto. Aceita nomes (`red`), hexadecimal, `rgb()` e `hsl()`.

```
h1 { color: #1B3A8C; }
```

#02 background-color — FUNDO

Cor de fundo do elemento.

```
body { background-color: #f5f5f5; }
```

#03 font-size — TEXTO

Tamanho da letra. Unidades comuns: `px`, `rem` e `em`.

```
p { font-size: 1.125rem; }
```

#04 font-family — TEXTO

Fonte do texto. Liste alternativas: se a 1ª não existir, usa a próxima.

```
body { font-family: "Poppins", Arial, sans-serif; }
```

#05 font-weight — TEXTO

Espessura da letra: `normal`, `bold` ou de 100 a 900.

```
strong { font-weight: 700; }
```

#06 font-style — TEXTO

Estilo da fonte: `normal` ou `italic`.

```
em { font-style: italic; }
```

#07 text-align — TEXTO

Alinhamento horizontal: `left`, `center`, `right`, `justify`.

```
h1 { text-align: center; }
```

#08 text-decoration — TEXTO

Linhas no texto: `underline`, `line-through` ou `none` (tira o sublinhado dos links).

```
a { text-decoration: none; }
```

#09 text-transform — TEXTO

Muda a caixa: `uppercase`, `lowercase`, `capitalize`.

```
.titulo { text-transform: uppercase; }
```

#10 line-height — TEXTO

Altura da linha. Sem unidade = multiplica o tamanho da fonte.

```
p { line-height: 1.6; }
```

#11 letter-spacing — TEXTO

Espaço entre as letras.

```
h2 { letter-spacing: 2px; }
```

#12 list-style — TEXTO

Marcadores de lista. `none` remove as bolinhas.

```
ul { list-style: none; }
```

#13 opacity — EFEITO

Transparência do elemento inteiro: de `0` (invisível) a `1`.

```
.apagado { opacity: 0.5; }
```

#14 cursor — INTERAÇÃO

Formato do ponteiro: `pointer` (mãozinha), `not-allowed`...

```
button { cursor: pointer; }
```

#15 text-shadow — TEXTO

Sombra no texto: X, Y, desfoque e cor.

```
h1 { text-shadow: 2px 2px 4px #0005; }
```

Nível 2 · A caixa: tamanho, espaço e bordas (Box Model)

#16 width · height — CAIXA

Largura e altura. Aceita `px`, `%`, `vw`, `vh`, `rem`...

```
.foto { width: 300px; height: 200px; }
```

#17 max-width · min-width — CAIXA

Limites de largura. Com `margin: auto` centraliza o site.

```
.container { max-width: 1200px; margin: 0 auto; }
```

#18 min-height · max-height — CAIXA

Altura mínima/máxima. `100vh` = altura da tela.

```
.hero { min-height: 100vh; }
```

#19 margin — ESPAÇO

Espaço EXTERNO. 1 a 4 valores no sentido horário: topo, direita, baixo, esquerda.

```
.card { margin: 10px 20px; }
```

#20 padding — ESPAÇO

Espaço INTERNO (entre conteúdo e borda). Mesma lógica da margin.

```
.botao { padding: 12px 24px; }
```

#21 border — BORDA

Espessura, estilo (`solid`, `dashed`, `dotted`) e cor.

```
.card { border: 2px solid #ddd; }
```

#22 border-radius — BORDA

Arredonda os cantos. `50%` num quadrado = círculo.

```
.avatar { border-radius: 50%; }
```

#23 box-sizing — CAIXA

Com `border-box`, a largura já inclui padding e borda.

```
* { box-sizing: border-box; }
```

#24 display — LAYOUT

Comportamento da caixa: `block`, `inline`, `inline-block`, `none`, `flex`, `grid`.

```
.menu li { display: inline-block; }
```

Dica do Músculo: Use `rem` nas fontes: respeita o tamanho que o usuário configurou.

Propriedades — do básico ao mais complexo

Nível 2 · A caixa: tamanho, espaço e bordas (Box Model) (continuação)

#25 box-shadow — EFEITO

Sombra da caixa: X, Y, desfoque, espalhamento e cor.

```
.card { box-shadow: 0 4px 12px #0002; }
```

#26 outline — BORDA

Contorno fora da borda, que não ocupa espaço. Essencial no foco.

```
input:focus { outline: 2px solid #2965F1; }
```

#27 visibility — EFEITO

`hidden` esconde mas continua ocupando espaço (≠ `display: none`).

```
.fantasma { visibility: hidden; }
```

#28 overflow — CAIXA

O que fazer quando não cabe: `hidden`, `scroll`, `auto`.

```
.caixa { overflow: auto; }
```

Nível 3 · Fundos, imagens e textos longos

#29 background-image — FUNDO

Imagem (ou degradê) de fundo.

```
.hero { background-image: url("praia.jpg"); }
```

#30 background-size — FUNDO

`cover` (cobre tudo), `contain` (cabe inteira) ou medidas.

```
.hero { background-size: cover; }
```

#31 background-position — FUNDO

Posição da imagem: `center`, `top`, `50% 20%`...

```
.hero { background-position: center; }
```

#32 background-repeat — FUNDO

Repetir ou não a imagem: `no-repeat`, `repeat-x`...

```
.hero { background-repeat: no-repeat; }
```

#33 background — FUNDO

Atalho que junta tudo de fundo em uma linha.

```
background: url(bg.jpg) center/cover no-repeat;
```

#34 object-fit — MÍDIA

Como `img`/`video` preenche a caixa sem distorcer.

```
img { height: 200px; object-fit: cover; }
```

#35 white-space — TEXTO

Controla quebras e espaços. `nowrap` impede a quebra.

```
.tag { white-space: nowrap; }
```

#36 text-overflow — TEXTO

Mostra "..." no texto cortado (com `overflow: hidden` e `nowrap`).

```
.titulo { text-overflow: ellipsis; }
```

#37 overflow-wrap · word-break — TEXTO

Quebra palavras gigantes (links, e-mails).

```
p { overflow-wrap: break-word; }
```

#38 vertical-align — TEXTO

Alinhamento vertical de itens em linha e células.

```
img { vertical-align: middle; }
```

Nível 4 · Posicionamento e camadas

#39 position — POSIÇÃO

`static`, `relative`, `absolute` (relativo ao pai posicionado), `fixed` (preso na tela), `sticky` (gruda ao rolar).

```
header { position: sticky; top: 0; }
```

#40 top · right · bottom · left — POSIÇÃO

Deslocamento de elementos posicionados.

```
.selo { position: absolute; top: 10px; right: 10px; }
```

#41 inset — POSIÇÃO

Atalho dos quatro lados. `inset: 0` estica no pai inteiro.

```
.overlay { position: fixed; inset: 0; }
```

#42 z-index — POSIÇÃO

Ordem das camadas: maior valor fica por cima.

```
.modal { z-index: 1000; }
```

#43 float · clear — POSIÇÃO

Faz o elemento "flutuar" com o texto ao redor.

```
img { float: left; margin-right: 1rem; }
```

Dica do Músculo: Centralizar? `display:flex; justify-content:center; align-items:center;`

Propriedades — do básico ao mais complexo

Nível 5 · Flexbox — layout em UMA direção

#44 display: flex
FLEXBOX

Vira um "container flex": os filhos ficam lado a lado e fáceis de alinhar.

```
.menu { display: flex; }
```

#45 flex-direction
FLEXBOX

Direção: `row` (linha, padrão) ou `column` (coluna).

```
.lista { flex-direction: column; }
```

#46 justify-content
FLEXBOX

Eixo principal: `center`, `space-between`, `space-around`, `space-evenly`...

```
nav { justify-content: space-between; }
```

#47 align-items
FLEXBOX

Eixo cruzado: `stretch`, `center`, `flex-start`, `flex-end`.

```
.linha { align-items: center; }
```

#48 flex-wrap
FLEXBOX

Deixa os itens quebrarem para a linha de baixo.

```
.galeria { flex-wrap: wrap; }
```

#49 gap
FLEX · GRID

Espaço entre os itens, sem precisar de margin.

```
.galeria { gap: 16px; }
```

#50 flex
FLEXBOX

Atalho de grow/shrink/basis. `flex: 1` ocupa o espaço livre.

```
.conteudo { flex: 1; }
```

#51 align-self
FLEXBOX

Alinha um item diferente dos outros.

```
.destaque { align-self: flex-end; }
```

#52 order
FLEXBOX

Muda a ordem visual sem mexer no HTML.

```
.primeiro { order: -1; }
```

#53 align-content
FLEXBOX

Distribui as LINHAS quando há quebra.

```
.galeria { align-content: space-between; }
```

Nível 6 · Grid — layout em DUAS direções

#54 display: grid
GRID

Transforma o elemento em uma grade de linhas e colunas.

```
.layout { display: grid; }
```

#55 grid-template-columns
GRID

Define as colunas. `fr` = fração do espaço livre.

```
grid-template-columns: 250px 1fr;
```

#56 grid-template-rows
GRID

Define as linhas da grade.

```
grid-template-rows: auto 1fr auto;
```

#57 repeat() · minmax()
GRID

Colunas repetidas e responsivas, sem media query!

```
repeat(auto-fit, minmax(200px, 1fr))
```

#58 grid-column · grid-row
GRID

Faz um item ocupar várias colunas/linhas.

```
.banner { grid-column: 1 / 3; }
```

#59 grid-template-areas
GRID

Desenha o layout com nomes, como um mapa.

```
grid-template-areas: "topo topo" "menu main";
```

#60 place-items
GRID

Atalho de alinhamento. `center` centraliza nos dois eixos.

```
.tela { display: grid; place-items: center; }
```

#61 aspect-ratio
CAIXA

Mantém a proporção da caixa (vídeo 16:9).

```
.video { aspect-ratio: 16 / 9; }
```

Nível 7 · Responsividade, variáveis e funções

#62 @media
RESPONSIVO

Aplica estilos só em certas condições (largura de tela, impressão...).

```
@media (max-width: 768px) {
  .menu { flex-direction: column; }
}
```

#63 --variavel · var()
VARIÁVEIS

Defina uma vez, reutilize no site todo.

```
:root { --azul: #1B3A8C; }
h1 { color: var(--azul); }
```

#64 calc()
FUNÇÃO

Faz contas misturando unidades.

```
width: calc(100% - 250px);
```

Dica do Músculo: Guarde as cores em variáveis: `--cor-primaria: #1B3A8C;`

Propriedades — do básico ao mais complexo

Nível 7 · Responsividade, variáveis e funções (continuação)

#65 clamp() · min() · max()
FUNÇÃO

`clamp(mín, ideal, máx)`: tamanhos fluidos com limites.

```
font-size: clamp(1.5rem, 4vw, 3rem);
```

#66 @font-face
AT-RULE

Carrega uma fonte personalizada de um arquivo.

```
@font-face { font-family: "Minha";
  src: url(minha.woff2); }
```

#67 @import
AT-RULE

Importa outro CSS (ou fontes do Google Fonts).

```
@import url("reset.css");
```

#68 @supports
AT-RULE

Só aplica se o navegador suportar a propriedade.

```
@supports (display: grid) { ... }
```

#69 @container
RESPONSIVO

Responde ao tamanho do PAI, não da tela.

```
.box { container-type: inline-size; }
@container (min-width: 400px) { ... }
```

#70 prefers-color-scheme
RESPONSIVO

Detecta se o usuário usa tema escuro.

```
@media (prefers-color-scheme: dark) { ... }
```

Nível 8 · Movimento e efeitos — os mais complexos de manipular

#71 transition
ANIMAÇÃO

Suaviza a mudança: propriedade, duração, curva e atraso.

```
.btn { transition: background .3s ease; }
```

#72 transform: translate()
TRANSFORMAÇÃO

Move o elemento em X/Y sem empurrar os vizinhos.

```
.btn:hover { transform: translateY(-3px);
}
```

#73 rotate() · scale() · skew()
TRANSFORMAÇÃO

Gira, aumenta/diminui e inclina.

```
transform: rotate(15deg) scale(1.2);
```

#74 transform-origin
TRANSFORMAÇÃO

O ponto de apoio (eixo) das transformações.

```
transform-origin: bottom center;
```

#75 @keyframes
ANIMAÇÃO

Define as etapas de uma animação (0% a 100%).

```
@keyframes pulsar {
  50% { transform: scale(1.1); }
}
```

#76 animation
ANIMAÇÃO

Aplica um `@keyframes`: nome, duração, curva, repetição.

```
animation: pulsar 1s ease infinite;
```

#77 linear-gradient()
FUNDO

Degradê em linha reta entre duas ou mais cores.

```
linear-gradient(90deg, #E34F26, #F7DF1E)
```

#78 radial-gradient() · conic-gradient()
FUNDO

Degradê circular ou em giro (ótimo para gráfico de pizza).

```
radial-gradient(circle, #fff, #1B3A8C)
```

#79 filter
EFEITO

Filtros: `blur()`, `grayscale()`, `brightness()`, `drop-shadow()`...

```
img:hover { filter: grayscale(100%); }
```

#80 backdrop-filter
EFEITO

Filtra o que está ATRÁS do elemento: vidro fosco.

```
.vidro { backdrop-filter: blur(10px); }
```

#81 clip-path
EFEITO

Recorta o elemento em formas (círculo, polígono).

```
clip-path: polygon(0 0, 100% 50%, 0
100%);
```

#82 mix-blend-mode
EFEITO

Mistura as cores com o fundo, como camadas do Photoshop.

```
.titulo { mix-blend-mode: difference; }
```

#83 content
PSEUDO-ELEMENTO

Conteúdo de `::before`/`::after`. Obrigatória neles.

```
.req::after { content: " *"; }
```

#84 pointer-events
INTERAÇÃO

`none` faz os cliques "atravessarem" o elemento.

```
.overlay { pointer-events: none; }
```

#85 user-select
INTERAÇÃO

Controla se o texto pode ser selecionado.

```
.botao { user-select: none; }
```

#86 scroll-behavior
INTERAÇÃO

Rolagem suave nos links internos (#âncoras).

```
html { scroll-behavior: smooth; }
```

#87 scroll-snap
INTERAÇÃO

Faz a rolagem "encaixar" nos itens (carrossel).

```
.carrossel { scroll-snap-type: x
mandatory; }
```

#88 accent-color
FORMULÁRIO

Cor de checkbox, radio e range nativos.

```
input { accent-color: #E34F26; }
```

Dica do Músculo: Mobile first: estilize o celular primeiro e use `@media (min-width)` para telas maiores.

Nível 8 · Movimento e efeitos — os mais complexos de manipular (continuação)

#89 perspective

3D

Dá profundidade para `rotateX()`/`rotateY()`.

```
.cena { perspective: 800px; }
```

#90 will-change

PERFORMANCE

Avisa o navegador do que vai animar.

```
.slide { will-change: transform; }
```

Dica do Músculo: Comentário em CSS: `/* assim */`

Como funcionam as cores em hexadecimal

Na tela, **toda cor é uma mistura de luz** de três canais: **Vermelho** (Red), **Verde** (Green) e **Azul** (Blue) — o famoso **RGB**. Cada canal vai de **0** (apagado) a **255** (aceso no máximo). O **hexadecimal** é só um jeito curto de escrever esses três números usando a **base 16**.



1 · Contando em base 16

Nós contamos de 0 a 9 (base 10). O hexadecimal conta até 15 com **um único dígito**, usando letras depois do 9:

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

2 · Anatomia de uma cor

1B 3A 8C ■

início | Red = 27 | Green = 58 | Blue = 140 | resultado

Sempre **3 pares** de dígitos: **#RRGGBB**. Cada par vai de `00` (0) a `FF` (255).
São 256 × 256 × 256 = **16.777.216 cores** possíveis!

3 · Convertendo na mão

Em cada par, o dígito da **esquerda vale ×16** e o da **direita vale ×1**:

`8C` → 8×16 + C(12) = 128 + 12 = **140**

`3A` → 3×16 + A(10) = 48 + 10 = **58**

`FF` → 15×16 + 15 = 240 + 15 = **255**

`00` → 0×16 + 0 = **0**

Na prática ninguém converte de cabeça: use o seletor de cor do VS Code ou do DevTools (F12) — mas agora você sabe o que está acontecendo!

4 · Cores para conhecer de cor

#000000 Preto (tudo apagado)	**#FFFFFF** Branco (tudo aceso)	**#FF0000** Vermelho puro	**#00FF00** Verde puro	**#0000FF** Azul puro	**#808080** Cinza médio
#FFFF00 Amarelo (R + G)	**#00FFFF** Ciano (G + B)	**#FF00FF** Magenta (R + B)	**#E34F26** Laranja do HTML	**#1B3A8C** Azul do CSS	**#F7DF1E** Amarelo do JS

Pares iguais = cinza. Quanto maior o número, mais claro:

00	22	44	66	88	AA	CC	EE	FF

Mexendo em um canal só (#**??**0000):

00	20	40	60	80	A0	C0	E0	FF

5 · Atalhos e transparência

3 dígitos: se os pares se repetem, dá para encurtar: #FF6600 = #F60, #FFFFFF = #FFF.
8 dígitos: o 4º par é a **opacidade** (alpha): #RRGGBBAA.

FF 100%	CC 80%	99 60%	80 50%	33 20%	00 0%

6 · Mesma cor, 3 formatos

■ **#E34F26**

= `rgb(227, 79, 38)`

= `hsl(13, 77%, 52%)`

Maiúsculas ou minúsculas tanto faz: `#e34f26` é a mesma cor.

Dica do Músculo: Com a mesma especificidade, a ÚLTIMA regra escrita vence.

JS

CAPÍTULO 3

JavaScript

O cérebro (e as ideias!) da página

O **JavaScript** é a linguagem de **programação** da web. Se o HTML é o esqueleto e o CSS a pele e os músculos, o JS é o **cérebro**: ele **pensa, decide e reage**. É ele que faz o menu abrir, o carrinho somar, o formulário avisar que o e-mail está errado e a página buscar dados sem recarregar.

⚡ Reagir

Escuta **eventos**: cliques, teclas, rolagem, envio de formulário.

🧠 Decidir

Usa **lógica**: variáveis, condições (`if`), repetições e funções.

🔧 Modificar

Altera o **DOM**: cria, remove e muda textos, classes e estilos.

🌐 Conversar

Busca e envia dados para servidores e **APIs** com `fetch`.

👆 Usuário clica
`evento "click"`

→

JS pensa
`if (logado) { ... }`

→

✨ Página muda
`classList.add("aberto")`

Neste capítulo

O DOM, como incluir o JS e o Console 27

Declaração de variáveis: var, let e const 28

Tipos de dados, typeof e conversões 29

Operadores: aritméticos, comparação e lógicos 30

Códigos JS essenciais (do mais importante ao mais avançado) 31

Lista de eventos mais usados 38

Projeto prático: lista de tarefas com HTML + CSS + JS 39

O que é o DOM?

Quando o navegador lê o seu HTML, ele monta na memória uma **árvore de objetos** chamada **DOM** (*Document Object Model*). Cada tag vira um "nó" dessa árvore, e o objeto `document` é a porta de entrada para todos eles.

O JavaScript **não mexe no arquivo .html**: ele mexe no DOM. E toda mudança no DOM aparece **na hora** na tela. Manipular o DOM é: **1)** encontrar o elemento, **2)** mudar alguma coisa nele, **3)** reagir a eventos.

```
document
└── html
    ├── head
    │   └── title
    └── body
        ├── h1 #titulo
        ├── ul .lista
        │   ├── li
        │   └── li
        └── button .btn   ◀ querySelector
```

Como colocar o JavaScript na página

EVITE

```
<button
onclick="alert('Oi')">
  Clique
</button>
```

No atributo: mistura HTML com lógica. Só para testes rápidos.

OK

```
<body>
  ...
  <script>
    console.log("Oi!");
  </script>
</body>
```

Interno: na tag `<script>`, no fim do body.

RECOMENDADO

```
<head>
  <script src="app.js"
          defer></script>
</head>
```

Externo + defer: arquivo separado, executa quando o HTML está pronto.

O Console: seu laboratório

Elements | **Console** | Sources | Network

```
> console.log("Olá, Work in Tech!")
  Olá, Work in Tech!
> 10 * 3 + 2
  32
> document.querySelector("h1").textContent
  'Minha primeira página'
✕ Uncaught ReferenceError: nome is not defined
                                  app.js:12
```

- Abra com `F12` (ou `Ctrl+Shift+J`) → aba **Console**.
- Dá para digitar JS direto ali e ver o resultado na hora.
- Os erros aparecem em **vermelho** com o **arquivo e a linha** do problema. Leia a mensagem: ela é sua amiga!

Os 3 passos para manipular o DOM

1 · ENCONTRAR

```
const btn = document
  .querySelector(".btn");
const nav = document
  .querySelector("nav");
```

2 · REAGIR

```
btn.addEventListener(
  "click", () => {
    // ao clicar...
  });
```

3 · MUDAR

```
nav.classList
  .toggle("aberto");
btn.textContent =
  "Fechar";
```

Dica do Cérebro: Use `const` por padrão e `let` só quando o valor for mudar. Esqueça o `var`.

Variáveis: as caixinhas de memória

Uma **variável** é uma **caixinha com etiqueta** onde o cérebro guarda uma informação para usar depois: um nome, uma idade, o total do carrinho, um elemento da página. **Declarar** é criar a caixinha; **atribuir** é colocar um valor dentro dela com o sinal `=`.

```
let   idade = 25;
```

palavra-chave | nome (etiqueta) | atribuição | valor | fim

Leia assim: "crie uma caixinha chamada **idade** e guarde o valor **25** nela". O = no JS **não** é "igual": é "recebe".

As 3 formas de declarar

Palavra	Pode mudar o valor?	Escopo	Redeclarar?	Quando usar
`const`	Não	Bloco `{ }`	Não	**Padrão!** Use para tudo que não será reatribuído (elementos, funções, configurações).
`let`	Sim	Bloco `{ }`	Não	Valores que mudam: contadores, totais, estados (`aberto = true`).
`var`	Sim	Função inteira	Sim (perigoso)	Jeito antigo (antes de 2015). Você vai ver em códigos velhos — **evite**.

LET E CONST

```
let pontos = 0;
pontos = pontos + 10;   // ✓ let pode mudar
pontos += 5;            // agora vale 15

const PI = 3.14;
PI = 3;   // ✗ TypeError: Assignment to
constant variable.

const frutas = ["maçã"];
frutas.push("uva");     // ✓ o CONTEÚDO pode
mudar
frutas = [];            // ✗ trocar a caixa
inteira, não
```

ESCOPO DE BLOCO

```
if (true) {
  let segredo = "só aqui dentro";
  var vazou = "escapei do bloco!";
}
console.log(vazou);    // "escapei do bloco!"
console.log(segredo);  // ✗ ReferenceError

let nome;              // declarada, sem
valor
console.log(nome);     // undefined
nome = "Ana";          // atribuída depois
```

Regras para dar nome

- Começa com letra, `_` ou `$` — nunca com número (`1nome` ✗).
- Sem espaços e sem hífen (`meu-nome` ✗). Evite acentos.
- Maiúsculas importam: `nome` e `Nome` são variáveis diferentes.
- Não pode usar palavras reservadas: `let`, `if`, `function`, `class`...

Boas práticas

- **camelCase:** `precoTotal`, `nomeDoUsuario`, `btnEnviar`.
- Nomes que explicam: `idadeMinima` ✓ x ✗
- Booleanos como pergunta: `estaLogado`, `temEstoque`.
- Constantes fixas em MAIÚSCULAS: `TAXA_ENTREGA`.

Dica do Cérebro: Prefira `===` a `==`: compara o valor E o tipo.

Tipos de dados: o que cabe na caixinha

O JavaScript tem **tipagem dinâmica**: você não diz o tipo da variável — ele descobre pelo valor, e o tipo pode mudar. Existem **tipos primitivos** (valores simples) e **objetos** (estruturas que agrupam valores). Use `typeof` para perguntar o tipo de algo.

String — PRIMITIVO

Texto. Entre aspas simples, duplas ou crase.

```
let nome = "Ana";
let msg = `Oi, ${nome}`; // template
```

Number — PRIMITIVO

Números inteiros e decimais (com ponto!). Inclui `NaN` e `Infinity`.

```
let idade = 25;
let preco = 19.90;
```

Boolean — PRIMITIVO

Verdadeiro ou falso. A base de todas as decisões.

```
let logado = true;
let maior = idade >= 18;
```

Undefined — PRIMITIVO

"Ainda não tem valor". É o padrão de variáveis sem atribuição.

```
let x;
console.log(x); // undefined
```

Null — PRIMITIVO

"Vazio de propósito". Você coloca para dizer "nada aqui".

```
let usuario = null;
```

BigInt · Symbol — PRIMITIVO

Inteiros gigantes (com `n` no fim) e identificadores únicos. Raros no dia a dia.

```
let big = 9007199254740993n;
let id = Symbol("id");
```

Object — OBJETO

Coleção de pares `chave: valor`. Representa "coisas".

```
let carro = { marca: "Fiat",
ano: 2020 };
carro.marca; // "Fiat"
```

Array — OBJETO

Lista ordenada. Índices começam em **0**.

```
let cores = ["red", "blue"];
cores[0]; // "red"
```

Function — OBJETO

Um bloco de código que também é um valor: pode ir dentro de variáveis.

```
const ola = () => "Olá!";
ola(); // "Olá!"
```

TYPEOF

```
typeof "oi"          // "string"
typeof 42            // "number"
typeof true          // "boolean"
typeof undefined     // "undefined"
typeof {}            // "object"
typeof []            // "object"  (array é objeto!)
typeof null          // "object"  (bug histórico 😅)
Array.isArray([])    // true ← jeito certo p/ arrays
```

CONVERSÕES E SURPRESAS

```
"5" + 1              // "51"  (+ com texto junta)
"5" - 1              // 4     (- converte p/ número)
Number("5") + 1      // 6     ✓ converta antes!
Number("abc")        // NaN   (Not a Number)
String(10)           // "10"
0.1 + 0.2            // 0.30000000000000004
"10" == 10           // true  (converte tipos)
"10" === 10          // false ✓ compara tipo também
```

Truthy e Falsy

Em um `if`, todo valor vira verdadeiro ou falso. Só **6 valores são falsy**: `false`, `0`, `""` (texto vazio), `null`, `undefined` e `NaN`. **Todo o resto é truthy** — inclusive `"0"`, `"false"`, `[]` e `{}`!

```
const nome = input.value;
if (nome) {
  // só entra se NÃO estiver vazio
}
```

Dica do Cérebro: `console.log()` é seu melhor amigo para descobrir o que está acontecendo.

Operadores: fazendo contas e comparações

Aritméticos

Op.	Faz	Exemplo
`+`	Soma (ou junta textos)	`5 + 2` → 7
`-`	Subtração	`5 - 2` → 3
`*`	Multiplicação	`5 * 2` → 10
`/`	Divisão	`5 / 2` → 2.5
`%`	Resto da divisão	`5 % 2` → 1 (ímpar!)
`**`	Potência	`2 ** 3` → 8
`++` `--`	Soma / tira 1	`i++`

Atribuição

Op.	Equivale a
`=`	Guarda o valor: `x = 10`
`+=`	`x = x + 5`
`-=`	`x = x - 5`
`*=` `/=`	`x = x * 2` · `x = x / 2`
`??=`	Atribui só se estiver `null`/`undefined`

Comparação (resultado: true/false)

Op.	Pergunta	Exemplo
`===`	É igual (valor e tipo)?	`5 === 5` → true
`!==`	É diferente?	`5 !== "5"` → true
`==`	Igual (converte tipo) ⚠	`5 == "5"` → true
`>` `<`	Maior / menor	`3 > 2` → true
`>=` `<=`	Maior/menor ou igual	`2 <= 2` → true

Lógicos

Op.	Significa	Exemplo
`&&`	E: tudo verdadeiro	`idade >= 18 && temCNH`
`||`	OU: pelo menos um	`sab || dom`
`!`	NÃO: inverte	`!logado`
`??`	Valor padrão se null/undef.	`nome ?? "Visitante"`
`?.`	Acessa só se existir	`user?.endereco?.rua`
`? :`	Ternário (if curto)	`ok ? "Sim" : "Não"`

NA PRÁTICA

```
const preco = 50;
const qtd = 3;
const frete = preco * qtd >= 100 ? 0 : 15;
const total = preco * qtd + frete;       // 150

const ehPar = (n) => n % 2 === 0;
ehPar(10);                               // true

const podeEntrar = idade >= 18 && !bloqueado;
const apelido = usuario.apelido || usuario.nome;
```

Ordem das contas: como na matemática — primeiro `**`, depois `* / %`, depois `+ -`. Na dúvida, use **parênteses**: `(2 + 3) * 4` = 20.

Lembre: `=` guarda, `===` compara. Trocar um pelo outro dentro de um `if` é um dos erros mais comuns!

Dica do Cérebro: Carregue seus scripts com `defer` ou no fim do `<body>`.

Códigos essenciais para manipular o DOM

1 · Mostrar informações e conversar com o usuário

#01 console.log()
SAÍDA

Mostra valores no Console do navegador (F12). É a ferramenta nº 1 para entender o que o seu código está fazendo.

```
let nome = "Ana";
console.log("Olá,", nome); // Olá, Ana
```

#02 console.error() · warn() · table()
SAÍDA

Variações do console: erro em vermelho, aviso em amarelo e arrays/objetos em formato de tabela.

```
console.error("Deu ruim!");
console.table([{ nome: "Ana", idade: 20 }]);
```

#03 alert()
JANELA

Abre uma caixa de aviso com um botão OK. A página fica travada até o usuário fechar.

```
alert("Cadastro realizado!");
```

#04 prompt()
JANELA

Abre uma caixa pedindo para o usuário digitar algo. Sempre devolve TEXTO (string) ou `null`.

```
let idade = prompt("Qual sua idade?");
```

#05 confirm()
JANELA

Pergunta com OK/Cancelar. Devolve `true` ou `false`.

```
if (confirm("Deseja excluir?")) {
  item.remove();
}
```

2 · Encontrar elementos na página (selecionar o DOM)

#06 document.getElementById()
SELEÇÃO

Pega UM elemento pelo id. Rápido e direto.

```
const titulo = document.getElementById("titulo");
```

#07 document.querySelector()
SELEÇÃO

Pega o PRIMEIRO elemento que bate com um seletor CSS (`.classe`, `#id`, `tag`...).

```
const btn = document.querySelector(".btn-comprar");
```

#08 document.querySelectorAll()
SELEÇÃO

Pega TODOS os elementos que batem com o seletor (uma NodeList, que aceita `forEach`).

```
const itens = document.querySelectorAll("li");
itens.forEach(li => console.log(li));
```

#09 getElementsByClassName()
SELEÇÃO

Todos os elementos com a classe, numa coleção "viva" que se atualiza sozinha.

```
const cards = document.getElementsByClassName("card");
```

3 · Ler e alterar o conteúdo

#10 .innerHTML
CONTEÚDO

Lê ou troca o HTML de dentro do elemento. As tags são interpretadas de verdade.

```
lista.innerHTML = "<li>Novo item</li>";
lista.innerHTML += "<li>Mais um</li>";
```

#11 .textContent
CONTEÚDO

Lê ou troca só o TEXTO (tags viram texto puro). Mais seguro e rápido para textos.

```
titulo.textContent = "Bem-vindo!";
```

#12 .innerText
CONTEÚDO

Parecido com textContent, mas respeita o CSS (ignora texto escondido).

```
console.log(paragrafo.innerText);
```

#13 .value
FORMULÁRIO

Lê ou define o valor de inputs, selects e textareas.

```
const email = inputEmail.value;
inputEmail.value = ""; // limpa o campo
```

#14 insertAdjacentHTML()
CONTEÚDO

Insere HTML numa posição exata sem apagar o que já existe: `beforebegin`, `afterbegin`, `beforeend`, `afterend`.

```
lista.insertAdjacentHTML("beforeend", "<li>Fim</li>");
```

Dica do Cérebro: JavaScript diferencia maiúsculas de minúsculas: `nome` ≠ `Nome`.

Códigos essenciais para manipular o DOM

4 · Classes CSS pelo JavaScript (o jeito certo de mudar o visual)

#15 classList.add()

CLASSES

Adiciona uma ou mais classes ao elemento.

```
menu.classList.add("aberto");
```

#16 classList.remove()

CLASSES

Remove uma ou mais classes do elemento.

```
menu.classList.remove("aberto");
```

#17 classList.toggle()

CLASSES

Liga/desliga: se tem a classe, remove; se não tem, adiciona. Perfeito para menus e modo escuro.

```
btn.addEventListener("click", () => {
  document.body.classList.toggle("dark");
});
```

#18 classList.contains()

CLASSES

Verifica se o elemento tem a classe. Devolve `true`/`false`.

```
if (menu.classList.contains("aberto")) { ... }
```

#19 classList.replace()

CLASSES

Troca uma classe por outra.

```
alerta.classList.replace("erro", "sucesso");
```

#20 .className

CLASSES

Lê ou substitui TODAS as classes de uma vez, como texto.

```
caixa.className = "card destaque";
```

5 · Estilos e atributos

#21 .style

ESTILO

Altera o CSS inline. Propriedades com hífen viram camelCase (`background-color` → `backgroundColor`).

```
caixa.style.backgroundColor = "#F7DF1E";
caixa.style.display = "none";
```

#22 getComputedStyle()

ESTILO

Lê o estilo FINAL aplicado ao elemento, venha de onde vier.

```
getComputedStyle(caixa).width; // "320px"
```

#23 setAttribute()

ATRIBUTOS

Cria ou altera um atributo do elemento.

```
img.setAttribute("src", "gato.png");
```

#24 getAttribute()

ATRIBUTOS

Lê o valor de um atributo.

```
const link = a.getAttribute("href");
```

#25 removeAttribute()

ATRIBUTOS

Remove um atributo.

```
botao.removeAttribute("disabled");
```

#26 .dataset

ATRIBUTOS

Acessa os atributos `data-*` do HTML como um objeto.

```
// <button data-produto-id="42">
btn.dataset.produtoId; // "42"
```

#27 .disabled · .hidden · .checked

ATRIBUTOS

Propriedades booleanas que espelham atributos do HTML.

```
botao.disabled = true;
aviso.hidden = false;
```

6 · Criar, mover e remover elementos

#28 document.createElement()

CRIAR

Cria um elemento novo na memória (ainda fora da página).

```
const li = document.createElement("li");
li.textContent = "Comprar pão";
```

#29 appendChild()

INSERIR

Coloca o elemento como ÚLTIMO filho de outro. Agora ele aparece na tela!

```
lista.appendChild(li);
```

Dica do Cérebro: Nomes de variáveis em camelCase: `precoTotal`, `nomeUsuario`.

Códigos essenciais para manipular o DOM

6 · Criar, mover e remover elementos (continuação)

#30 append() · prepend()
INSERIR

Insere no fim (`append`) ou no início (`prepend`). Aceitam vários elementos e textos.

```
lista.prepend(primeiroItem);
div.append("Texto", span);
```

#31 before() · after()
INSERIR

Insere elementos antes ou depois, como irmãos.

```
titulo.after(subtitulo);
```

#32 .remove()
REMOVER

Remove o próprio elemento da página.

```
aviso.remove();
```

#33 removeChild()
REMOVER

Jeito antigo: o pai remove um filho.

```
lista.removeChild(lista.firstElementChild);
```

#34 cloneNode()
CRIAR

Copia um elemento. Com `true`, copia tudo que está dentro também.

```
const copia = card.cloneNode(true);
```

#35 replaceWith()
INSERIR

Substitui o elemento por outro.

```
antigo.replaceWith(novo);
```

7 · Navegar pela árvore do DOM

#36 .parentElement
NAVEGAÇÃO

O elemento pai (quem está "por fora").

```
btn.parentElement.remove(); // remove o card
```

#37 .children
NAVEGAÇÃO

Coleção com os elementos filhos.

```
console.log(lista.children.length);
```

#38 closest()
NAVEGAÇÃO

Sobe pela árvore até achar o ancestral mais próximo que bate com o seletor.

```
const card = event.target.closest(".card");
```

#39 nextElementSibling · previousElementSibling
NAVEGAÇÃO

O próximo / o anterior irmão (mesmo nível).

```
titulo.nextElementSibling.classList.add("intro");
```

#40 firstElementChild · lastElementChild
NAVEGAÇÃO

Primeiro e último filho.

```
lista.lastElementChild.remove();
```

8 · Eventos: fazendo a página reagir

#41 addEventListener()
EVENTOS

"Escuta" um evento (clique, digitação, envio...) e executa uma função quando ele acontece.

```
btn.addEventListener("click", () => {
  alert("Clicou!");
});
```

#42 removeEventListener()
EVENTOS

Para de escutar. Precisa receber a MESMA função (com nome) usada no add.

```
btn.removeEventListener("click", aoClicar);
```

#43 event (o objeto do evento)
EVENTOS

A função recebe um objeto com os detalhes: elemento, tecla, posição do mouse...

```
input.addEventListener("keydown", (e) => {
  console.log(e.key); // tecla pressionada
});
```

#44 event.target
EVENTOS

O elemento exato que disparou o evento.

```
document.addEventListener("click", (e) => {
  console.log(e.target);
});
```

Dica do Cérebro: `textContent` é mais seguro que `innerHTML` para textos digitados pelo usuário.

Códigos essenciais para manipular o DOM

8 · Eventos: fazendo a página reagir (continuação)

#45 event.preventDefault()

EVENTOS

Cancela o comportamento padrão: o form não recarrega a página, o link não navega...

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
});
```

#46 event.stopPropagation()

EVENTOS

Impede que o evento "suba" para os elementos pais.

```
modal.addEventListener("click", e => e.stopPropagation());
```

#47 DOMContentLoaded

EVENTOS

Dispara quando o HTML terminou de carregar: garante que os elementos já existem.

```
document.addEventListener("DOMContentLoaded", iniciar);
```

#48 Delegação de eventos

EVENTOS

Um único listener no pai cuida de todos os filhos, até dos criados depois!

```
lista.addEventListener("click", (e) => {
  if (e.target.matches(".excluir"))
    e.target.closest("li").remove();
});
```

#49 .onclick (propriedade)

EVENTOS

Forma curta de ligar UM evento. Atribuir de novo substitui o anterior.

```
btn.onclick = () => console.log("oi");
```

9 · Lógica: funções, decisões e repetições

#50 function

FUNÇÕES

Bloco de código reutilizável com nome. Recebe parâmetros e devolve um valor com `return`.

```
function somar(a, b) {
  return a + b;
}
somar(2, 3); // 5
```

#51 Arrow function () =>

FUNÇÕES

Forma curta de escrever funções. Muito usada em eventos e métodos de array.

```
const dobro = (n) => n * 2;
dobro(4); // 8
```

#52 if · else if · else

DECISÃO

Executa um bloco apenas se a condição for verdadeira.

```
if (nota >= 7) {
  status = "Aprovado";
} else if (nota >= 5) {
  status = "Recuperação";
} else { status = "Reprovado"; }
```

#53 Ternário ? :

DECISÃO

Um if/else em uma linha: `condição ? seSim : seNão`.

```
const msg = idade >= 18 ? "Maior" : "Menor";
```

#54 switch

DECISÃO

Compara um valor com vários casos. Não esqueça o `break`!

```
switch (dia) {
  case "sab": case "dom": console.log("Folga!"); break;
  default: console.log("Trabalho");
}
```

#55 for

REPETIÇÃO

Repete um número definido de vezes: início; condição; incremento.

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

#56 while

REPETIÇÃO

Repete ENQUANTO a condição for verdadeira. Cuidado com o loop infinito!

```
let n = 3;
while (n > 0) { console.log(n); n--; }
```

#57 for...of

REPETIÇÃO

Percorre cada item de um array (ou string, NodeList...).

```
for (const fruta of frutas) {
  console.log(fruta);
}
```

Dica do Cérebro: Comentários: `// uma linha` ou `/* várias linhas */`

Códigos essenciais para manipular o DOM

9 · Lógica: funções, decisões e repetições (continuação)

#58 try · catch — ERROS

Tenta executar e, se der erro, trata sem quebrar a página.

```
try {
  JSON.parse(textoInvalido);
} catch (erro) { console.error(erro.message); }
```

10 · Arrays e textos

#59 Arrays [ ] e .length — ARRAYS

Listas de valores. O índice começa em 0. `.length` diz quantos itens há.

```
const frutas = ["maçã", "uva"];
frutas[0];      // "maçã"
frutas.length;  // 2
```

#60 push() · pop() · unshift() · shift() — ARRAYS

Adiciona no fim (`push`) ou no início (`unshift`); remove do fim (`pop`) ou do início (`shift`).

```
frutas.push("kiwi");
frutas.shift();
```

#61 forEach() — ARRAYS

Executa uma função para cada item do array.

```
frutas.forEach((f, i) => console.log(i, f));
```

#62 map() — ARRAYS

Cria um NOVO array transformando cada item. Ótimo para gerar HTML a partir de dados.

```
const html = frutas.map(f => `<li>${f}</li>`).join("");
```

#63 filter() — ARRAYS

Cria um novo array só com os itens que passam no teste.

```
const caros = produtos.filter(p => p.preco > 100);
```

#64 find() · findIndex() — ARRAYS

Devolve o PRIMEIRO item (ou o índice dele) que passa no teste.

```
const ana = usuarios.find(u => u.nome === "Ana");
```

#65 includes() · indexOf() — ARRAYS · TEXTO

Verifica se contém um valor / em que posição ele está (-1 se não achar).

```
frutas.includes("uva"); // true
```

#66 reduce() — ARRAYS

Reduz o array a um único valor, como a soma de um carrinho.

```
const total = carrinho.reduce((s, p) => s + p.preco, 0);
```

#67 sort() · reverse() — ARRAYS

Ordena e inverte (altera o original!). Para números, passe uma função.

```
numeros.sort((a, b) => a - b);
```

#68 join() · split() — ARRAYS · TEXTO

`join` junta um array em texto; `split` quebra um texto em array.

```
"a,b,c".split(",");      // ["a","b","c"]
["a","b"].join(" - ");   // "a - b"
```

#69 Template literals `${ }` — TEXTO

Textos com crase aceitam variáveis e quebras de linha.

```
const msg = `Olá, ${nome}! Você tem ${idade} anos.`;
```

#70 toUpperCase() · toLowerCase() · trim() — TEXTO

Maiúsculas, minúsculas e remoção dos espaços nas pontas.

```
"  Oi  ".trim().toUpperCase(); // "OI"
```

#71 replace() · slice() · length — TEXTO

Troca trechos, recorta partes e conta caracteres.

```
"JavaScript".slice(0, 4);    // "Java"
"pão".replace("ã", "a");     // "pao"
```

Dica do Cérebro: Prefira `addEventListener` ao atributo `onclick` no HTML.

Códigos essenciais para manipular o DOM

11 · Números, matemática e datas

#72 Number() · parseInt() · parseFloat()

NÚMEROS

Convertem texto em número. Essencial com valores de inputs e do prompt (que são texto).

```
Number("10") + 5;     // 15
parseInt("42px");     // 42
parseFloat("3.14");   // 3.14
```

#73 toFixed()

NÚMEROS

Formata com casas decimais fixas (devolve texto).

```
(9.5).toFixed(2); // "9.50"
```

#74 toLocaleString()

NÚMEROS

Formata números e moedas no padrão do país.

```
(1500).toLocaleString("pt-BR",
  { style: "currency", currency: "BRL" }); // "R$ 1.500,00"
```

#75 Math.random() · Math.floor()

MATEMÁTICA

`random` sorteia de 0 a 1; `floor` arredonda pra baixo. Juntos: inteiro aleatório.

```
const dado = Math.floor(Math.random() * 6) + 1;
```

#76 Math.round() · ceil() · max() · min()

MATEMÁTICA

Arredonda, arredonda pra cima e acha o maior/menor valor.

```
Math.max(3, 9, 1); // 9
```

#77 new Date()

DATAS

Cria um objeto com a data e hora atuais (ou de uma data específica).

```
const hoje = new Date();
hoje.getFullYear();               // 2026
hoje.toLocaleDateString("pt-BR"); // "14/10/2026"
```

12 · Tempo: agendar e repetir

#78 setTimeout()

TEMPO

Executa uma função UMA vez depois de um tempo (em milissegundos).

```
setTimeout(() => aviso.remove(), 3000); // 3 s
```

#79 setInterval()

TEMPO

Executa repetidamente a cada intervalo. Ótimo para relógios e carrosséis.

```
const id = setInterval(atualizarRelogio, 1000);
```

#80 clearInterval() · clearTimeout()

TEMPO

Cancela um intervalo/timeout usando o id que ele devolveu.

```
clearInterval(id);
```

13 · Objetos, dados e armazenamento

#81 Objetos { }

DADOS

Agrupam dados em pares `chave: valor`. Acesse com ponto ou colchetes.

```
const user = { nome: "Ana", idade: 20 };
user.nome;      // "Ana"
user["idade"];  // 20
```

#82 Desestruturação

DADOS

Extrai valores de objetos e arrays para variáveis em uma linha.

```
const { nome, idade } = user;
const [primeiro, segundo] = frutas;
```

#83 Spread ...

DADOS

"Espalha" os itens de um array/objeto. Útil para copiar e juntar.

```
const todas = [...frutas, "manga"];
const copia = { ...user, idade: 21 };
```

#84 JSON.stringify()

DADOS

Transforma objeto/array em texto JSON (para salvar ou enviar).

```
JSON.stringify({ a: 1 }); // '{"a":1}'
```

#85 JSON.parse()

DADOS

Transforma texto JSON de volta em objeto/array.

```
const obj = JSON.parse('{"a":1}');
```

#86 localStorage

ARMAZENAMENTO

Guarda dados no navegador mesmo depois de fechar a página (só texto!).

```
localStorage.setItem("tarefas", JSON.stringify(lista));
const salvas = JSON.parse(localStorage.getItem("tarefas")) ||
[];
localStorage.removeItem("tarefas");
```

Dica do Cérebro: Funções pequenas, que fazem uma coisa só, são mais fáceis de entender e testar.

Códigos essenciais para manipular o DOM

13 · Objetos, dados e armazenamento (continuação)

#87 sessionStorage

ARMAZENAMENTO

Igual ao localStorage, mas tudo some ao fechar a aba.

```
sessionStorage.setItem("passo", "2");
```

14 · Internet e código assíncrono

#88 fetch()

REDE

Busca dados de uma URL/API. Devolve uma Promise com a resposta.

```
fetch("https://api.exemplo.com/users")
  .then(res => res.json())
  .then(dados => console.log(dados));
```

#89 async · await

ASSÍNCRONO

Escreve código assíncrono como se fosse "normal": o `await` espera a Promise terminar.

```
async function carregar() {
  const res = await fetch("/api/produtos");
  const produtos = await res.json();
}
```

#90 Promise · then() · catch()

ASSÍNCRONO

Promessa de um valor futuro. `then` recebe o resultado; `catch` trata o erro.

```
fetch(url).then(r => r.json())
  .catch(err => alert("Falhou!"));
```

15 · Janela, rolagem e foco

#91 window.location

NAVEGAÇÃO

Informações da URL atual e redirecionamento.

```
location.href = "obrigado.html"; // redireciona
location.reload();               // recarrega
```

#92 scrollIntoView() · scrollTo()

ROLAGEM

Rola a página até um elemento ou até uma posição.

```
secao.scrollIntoView({ behavior: "smooth" });
window.scrollTo({ top: 0 });
```

#93 focus() · blur()

FOCO

Coloca ou tira o foco de um campo.

```
inputNome.focus();
```

#94 window.innerWidth · innerHeight

JANELA

Largura e altura visíveis da janela, em pixels.

```
if (window.innerWidth < 768)
console.log("Celular!");
```

#95 dialog.showModal() · close()

INTERFACE

Abre/fecha um `<dialog>` como modal nativo, com fundo escurecido.

```
document.querySelector("dialog").showModal();
```

Dica do Cérebro: Deu erro? Abra o Console (F12): a mensagem diz o problema e a linha!

Eventos: tudo que a página consegue escutar

Um **evento** é algo que acontece na página: um clique, uma tecla, a rolagem. Com `elemento.addEventListener("nome-do-evento", funcao)` o cérebro fica "de ouvido" e reage. Estes são os eventos que você mais vai usar:

Evento	Dispara quando...	Uso típico
🖱 **Mouse**		
`click`	O elemento é clicado	Botões, menus, curtir
`dblclick`	Clique duplo	Editar um item
`mouseenter · mouseleave`	O mouse entra / sai do elemento	Tooltips, efeitos de destaque
`mousemove`	O mouse se move sobre o elemento	Desenhos, efeitos que seguem o cursor
`contextmenu`	Clique com o botão direito	Menu personalizado
⌨ **Teclado**		
`keydown`	Uma tecla é pressionada (`e.key`)	Atalhos, Enter para enviar, jogos
`keyup`	A tecla é solta	Contar caracteres digitados
📝 **Formulários**		
`submit`	O formulário é enviado	Validar e usar `preventDefault()`
`input`	O valor muda a cada letra digitada	Busca ao vivo, contador
`change`	O valor muda e o campo perde o foco	Select, checkbox, upload
`focus · blur`	O campo ganha / perde o foco	Destacar campo, validar ao sair
`reset`	O formulário é limpo	Restaurar estado
◻ **Janela e documento**		
`DOMContentLoaded`	O HTML terminou de carregar	Iniciar o script
`load`	Tudo carregou (imagens inclusive)	Esconder tela de "carregando"
`scroll`	A página (ou elemento) rola	Header que muda, botão "voltar ao topo"
`resize`	A janela muda de tamanho	Ajustes responsivos
`beforeunload`	O usuário vai sair da página	"Tem certeza? Há dados não salvos"
📱 **Toque e outros**		
`touchstart · touchend`	O dedo toca / sai da tela	Gestos no celular
`dragstart · drop`	Arrastar e soltar elementos	Kanban, upload arrastando
`copy · paste`	Copiar / colar	Avisar "copiado!"
`transitionend · animationend`	Uma transição/animação CSS termina	Remover elemento após sumir

OBJETO DO EVENTO

```
campo.addEventListener("keydown", (e) => {
  console.log(e.key);       // tecla: "Enter", "a", "ArrowUp"...
  console.log(e.target);    // o elemento que disparou
  if (e.key === "Enter") enviar();
});
```

Dica do Cérebro: Valores de `input` e `prompt` são texto: converta com `Number()` antes de somar.

Projeto prático: lista de tarefas (parte 1)

Hora de juntar **esqueleto, músculos e cérebro**! Vamos criar uma lista de tarefas onde dá para **adicionar**, **marcar como feita** e **excluir** itens — e que fica salva mesmo fechando o navegador. Crie 3 arquivos na mesma pasta:

INDEX.HTML — ESQUELETO

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Minhas tarefas</title>
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>
  <main class="app">
    <h1>Minhas tarefas</h1>
    <form id="form">
      <input id="campo" placeholder="O que
fazer?" required>
      <button>Adicionar</button>
    </form>
    <ul id="lista"></ul>
    <p id="contador">0 tarefas</p>
  </main>
</body>
</html>
```

STYLE.CSS — MÚSCULOS

```css
* { box-sizing: border-box; }
body {
  font-family: Arial, sans-serif;
  background: #F5F8FF;
  display: grid;
  place-items: center;
  min-height: 100vh;
}
.app {
  width: min(400px, 90%);
  background: #fff;
  padding: 24px;
  border-radius: 12px;
  box-shadow: 0 8px 24px #0001;
}
form { display: flex; gap: 8px; }
input { flex: 1; padding: 10px; }
button { background: #1B3A8C; color:
#fff;
  border: 0; padding: 10px 14px;
  border-radius: 6px; cursor: pointer; }
li { display: flex; justify-content:
space-between;
  padding: 10px 0; border-bottom: 1px
solid #eee; }
li.feita span { text-decoration: line-
through;
  opacity: .5; }
```

RESULTADO NO NAVEGADOR



HTML + CSS + JS = página viva!

Dica do Cérebro: Use `const` por padrão e `let` só quando o valor for mudar. Esqueça o `var`.

Projeto prático: lista de tarefas (parte 2 — o cérebro)

APP.JS — CÉREBRO

```
// 1. Encontrar os elementos (1)
const form = document.querySelector("#form");
const campo = document.querySelector("#campo");
const lista = document.querySelector("#lista");
const contador = document.querySelector("#contador");

// 2. Carregar tarefas salvas (ou começar vazio) (2)
let tarefas = JSON.parse(localStorage.getItem("tarefas")) ||
[];

// 3. Desenhar a lista na tela (3)
function renderizar() {
  lista.innerHTML = "";
  tarefas.forEach((tarefa, i) => {
    const li = document.createElement("li");
    li.classList.toggle("feita", tarefa.feita);
    li.dataset.index = i;
    li.innerHTML = `<span>${tarefa.texto}</span>
                    <button class="excluir">🗑</button>`;
    lista.appendChild(li);
  });
  const pendentes = tarefas.filter(t => !t.feita).length;
  contador.textContent = `${pendentes} tarefas pendentes`;
  localStorage.setItem("tarefas", JSON.stringify(tarefas));
}

// 4. Adicionar ao enviar o formulário (4)
form.addEventListener("submit", (e) => {
  e.preventDefault();                // não recarrega a página
  tarefas.push({ texto: campo.value.trim(), feita: false });
  campo.value = "";
  campo.focus();
  renderizar();
});

// 5. Marcar ou excluir (delegação de eventos) (5)
lista.addEventListener("click", (e) => {
  const li = e.target.closest("li");
  if (!li) return;
  const i = Number(li.dataset.index);
  if (e.target.matches(".excluir")) {
    tarefas.splice(i, 1);            // remove do array
  } else {
    tarefas[i].feita = !tarefas[i].feita;   // inverte
  }
  renderizar();
});

renderizar(); // 6. Desenha ao abrir a página (6)
```

Entendendo o cérebro

1 · Selecionar
`querySelector` guarda os elementos em constantes para usar depois.

2 · Memória
`localStorage` só guarda texto: por isso `JSON.parse` ao ler e `JSON.stringify` ao salvar. O `|| []` evita erro na 1ª visita.

3 · Renderizar
Limpa a lista e recria cada `li` a partir do array. O array é a "verdade"; a tela só reflete ele.

4 · Evento submit
`preventDefault()` impede o recarregamento; `push` adiciona; `trim()` tira espaços.

5 · Delegação
Um único listener na `ul` cuida de todos os itens — até dos criados depois. `closest` acha o `li` clicado.

6 · Início
Chamar `renderizar()` desenha as tarefas salvas assim que a página abre.

Desafios para treinar

- Não deixar adicionar tarefa vazia (use `if`).
- Criar um botão "Limpar concluídas" (use `filter`).
- Adicionar um modo escuro com `classList.toggle`.
- Editar a tarefa com duplo clique (`dblclick`).

Dica do Cérebro: Prefira `===` a `==`: compara o valor E o tipo.

FIM DO DICIONÁRIO · COMEÇO DA JORNADA

Agora a página está viva.

Você conheceu o **esqueleto** que estrutura, a **pele e os músculos** que dão forma e beleza, e o **cérebro** que pensa e reage. Volte a este guia sempre que precisar — e lembre: todo desenvolvedor consulta documentação todos os dias.



HTML

ESTRUTURA

Diz *o que* cada coisa é: títulos, textos, imagens, links e formulários.



CSS

APARÊNCIA

Diz *como* tudo fica: cores, fontes, espaços, layout e animações.



JavaScript

COMPORTAMENTO

Diz *o que acontece*: reage a cliques, decide e muda a página.

WORK in TECH

DICIONÁRIO DOS WEB FUNDAMENTOS · BONS ESTUDOS!